IJRR

Scenario-Based Motion Planning with Bounded Probability of Collision

Oscar de Groot, Laura Ferranti, Dariu Gavrila, Javier Alonso-Mora

arXiv:2307.01070v1 [cs.RO] 3 Jul 2023

***Abstract*—Robots will increasingly operate near humans that introduce uncertainties in the motion planning problem due to their complex nature. Typically, chance constraints are introduced in the planner to optimize performance while guaranteeing probabilistic safety. However, existing methods do not consider the actual probability of collision for the planned trajectory, but rather its marginalization, that is, the independent collision probabilities for each planning step and/or dynamic obstacle, resulting in conservative trajectories. To address this issue, we introduce a novel real-time capable method termed Safe Horizon MPC, that explicitly constrains the joint probability of collision with all obstacles over the duration of the motion plan. This is achieved by reformulating the chance-constrained planning problem using scenario optimization and predictive control. Our method is less conservative than state-of-the-art approaches, applicable to arbitrary probability distributions of the obstacles' trajectories, computationally tractable and scalable. We demonstrate our proposed approach using a mobile robot and an autonomous vehicle in an environment shared with humans.**

***Index Terms*—Motion and Path Planning, Optimization and Optimal Control, Collision Avoidance, Probability and Statistical Methods**

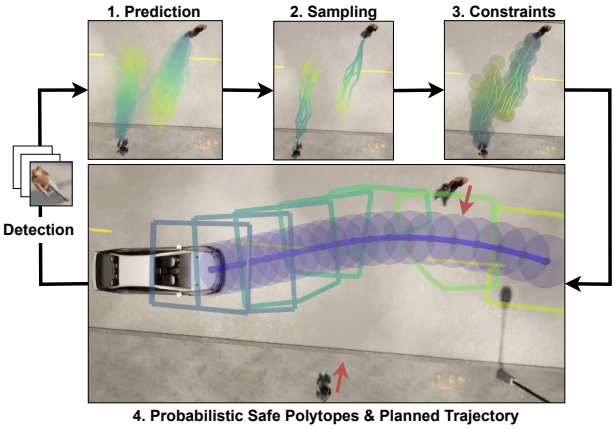


Fig. 1: Overview of the proposed scenario-based motion planner. Predicted obstacle trajectories are sampled to obtain scenarios, where each scenario represents a trajectory for all obstacles over the planning horizon. By ensuring safety for all scenarios, probabilistic safety of the motion plan is guaranteed.

I. Introduction

MOBILE robots can improve our quality of life, from transporting goods in warehouses [1] to helping us commute more efficiently and safely using self-driving vehicles [2]. In most applications where robots are currently deployed, the operating domain does not allow the robots to operate near humans (e.g., using dedicated lanes) to simplify the robot-navigation task. To deploy mobile robots in real-world environments, such as our cities, they need to be capable of navigating among humans, which remains challenging.

In crowded environments, the robot needs to understand and infer the motion of humans in order to move safely and efficiently. Unfortunately, human behavior is hard to predict and varies per person. In addition, human intentions cannot be explicitly communicated to the robot. This inherently makes the motion prediction of humans non deterministic. Recent perception methods, such as [3], infer human intentions, returning probabilistic information concerning humans. These probabilistic predictions need to be considered and exploited by the motion planner.

The authors are with the Dept. of Cognitive Robotics, TU Delft, 2628 CD Delft, The Netherlands. Email: o.m.degroot@tudelft.nl

This work received support from the Dutch Science Foundation NWO-TTW, within the SafeVRU project (14667) and Veni project HARMONIA (18165), and the European Union, within the ERC Starting Grant INTERACT (101041863). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Research Council Executive Agency. Neither the European Union nor the granting authority can be held responsible for them.

In a probabilistic setting, the occurrence of a collision is a *probabilistic* event. Our goal is to assess and bound the *Collision Probability* (CP) of the planned trajectory. This is challenging because of the non-Gaussian and multi-modal, i.e., multiple paths are possible, distributions involved when predicting human behavior. In addition, when the collision probability spans a duration, e.g., over the planned trajectory, then it must consider the correlation over time. The time correlation exists because the first collision in a trajectory renders it unsafe, such that all subsequent collisions can be ignored. Almost all of previous works, e.g., [4]–[10], do not account for this correlation, which leads to conservative trajectories. Similarly, the collision probability in these works is computed per obstacle, which degrades performance when more obstacles influence the plan, i.e., in crowded environments. By accounting for the correlation in time and by considering all obstacles, one would be able to plan more efficient trajectories, without compromising safety.

We achieve this goal by devising a novel probabilistic safe motion planner based on scenario optimization and nonlinear Model Predictive Control (MPC). Nonconvex Scenario optimization [11] is a sampling based approach for assessing the probability that a constraint holds under uncertainty. In the context of this paper, the imposed constraint probabilistically enforces collision avoidance between the controlled robot and the dynamic obstacles over the duration of the motion plan (see Fig. 1). The uncertainty associated with the motion of detected obstacles is predicted forward in time (Step 1). We sample

scenarios from these predictions that describe the trajectories of all dynamic obstacles during the planning horizon (Step 2) and construct collision avoidance constraints around each of the scenarios (Step 3). The robot trajectory is optimized with respect to the constraints (Step 4) which provides guaranteed probabilistic collision avoidance. The exact guarantees are tied to the number of sampled scenarios, which we determine before deploying the controller. This is, to our knowledge, the first real-time capable method that directly provides probabilistic safety guarantees on the planned trajectory, in contrast with the existing state-of-the-art where the same quantity is conservatively approximated through its marginals. We refer to our novel probabilistic safe motion planner as *Safe Horizon MPC* (SH-MPC).

We compare our method, for which we provide theoretical guarantees, to several baselines, in multiple scenarios, both with mobile robots and self-driving vehicles, and for varied probability distributions. We show that our planner is less conservative and improves efficiency, in terms of traveling time, while remaining safe.

II. Related Work and Contribution

A sizable body of literature exists on motion planning for safe autonomous navigation. In this section, we review some of the existing work on trajectory optimization with collision avoidance under uncertainty. For a general overview of autonomous navigation, we refer the reader to [12] and [13].

A. Collision Avoidance under Uncertainty

An important problem in autonomous navigation is to prevent collisions with dynamic obstacles. In trajectory optimization, the navigation problem is formulated as an optimization problem where performance criteria are optimized (e.g., lane following and progress towards the goal) under constraints (e.g., collision avoidance and actuator limits) [14], [15]. Due to large and multi-modal uncertainties in human motion prediction, collision avoidance in the mean or nominal case (e.g., constant velocity) may lead to collisions in practice. Many works therefore consider how to address this uncertainty.

One can consider the collision avoidance problem as a special case of optimization under uncertainty, for which two common approaches exist. *Robust optimization* [16] requires the constraints to be satisfied for all possible realizations of the uncertainty, while *stochastic optimization* (see [17] for an overview) allows for violation of the constraints, as long this happens with a probability smaller than an upper bound ϵ. Because the set of all possible realizations is often not available or too conservative, we focus in the remainder of this section on stochastic optimization. We refer to [18] for more details on both methods in the context of Model Predictive Control (MPC).

B. Collision Avoidance Chance Constraints

The probability of constraint violation in stochastic optimization is specified through *chance constraints*, which constrain the *probability* that a nominal constraint is satisfied. Exact evaluation of these chance constraints is however intractable in almost all applications. Existing works therefore focus on an approximation of the constraints, through additional assumptions on the probability distribution associated with the uncertainty (e.g., Gaussian [4], [19]) or on the controlled robot (e.g., linear dynamics [6]). Recent works [9], [10] have resolved many of the assumptions, making the framework applicable to nonlinear robot dynamics and arbitrary probability distributions. However, the chance constraints in these and many other works are not imposed on the robot's planned trajectory, i.e., the timed sequence of planned positions, but rather on each of its individual positions over the planning horizon. This fails to accurately bound the probability of colliding at *any* time[1]. In this regard, three types of chance constraint formulations have been considered in previous work: *Marginal*, *Conditional* and *Joint*, summarized in Table I.

Method	Non Gaussian	Dynamics	Computation Times*
Additive Marginal [4]–[6], [20]		Linear	10 ms
Multiplicative Marginal [7]		Linear	100 ms
Additive Marginal [8]–[10]	X	Nonlinear	10 ms
Conditional Marginal [21]		Linear	10 ms
Joint Trajectory [22], [23]	X	Linear	1000 ms
Joint Trajectory [24] (Estimation only)	X	Nonlinear	1000 ms
Joint Trajectory **Ours**	**X**	**Nonlinear**	**10 ms**

TABLE I: Comparison of motion planning method with collision avoidance chance constraints. *Order of magnitude, exact times vary with the acceptable CP, application and computational hardware.

Marginal Chance Constraints

Constraints on each position along the trajectory are referred to as *marginal* chance constraints. Let event A_k denote the case that no collisions occur at time k and $\mathbb{P}(A_k)$ therefore be the probability that the robot is safe at timestep k. Then the exact probability that a trajectory is safe over N steps is given by $\mathbb{P}(A) = \prod_{k=1}^{N} \mathbb{P}(A_k \mid A_{0:k-1})$. That is, the CP for each position is conditional on the probability of avoiding collisions up until the position is reached at time k. The problem is simplified if we assume instead that this event is independent for all states ($\tilde{\mathbb{P}}(A) \approx \prod_k \mathbb{P}(A_k)$). In [23], these marginal methods are further divided into *additive* and *multiplicative* approaches.

Additive approaches impose constraints on each marginal probability (i.e., $\mathbb{P}(A_k) \leq \epsilon_k$). Using Boole's inequality ($\mathbb{P}(\cup_k A_k) \leq \sum_k \mathbb{P}(A_k)$) the CP of the trajectory is bounded by the sum of the individual CPs. Under Gaussian uncertainty, the work [6] reformulated the constraints as an analytical constraint on the 1D Cumulative Density Function (CDF). The same idea is used in [4] in an MPC framework to prevent

[1]Similarly, chance constraints imposed per obstacle fail to estimate the probability of colliding with *any* obstacle.

collision between robot and obstacle volumes. In [20] the bound on each marginal probability is updated, known as *risk allocation*, while maintaining the same total risk bound (i.e., $\sum_k \epsilon_k = \epsilon$). In [5], [25] marginal chance constraints are applied to the Rapidly expanding Random Trees (RRT) algorithm such that each node in the tree is statistically safe. When the uncertainty is non Gaussian, the CDF of the probability distribution is typically not available. In [8], [9] an MPC for motion planning is formulated where inequalities are posed on stochastic moments of the marginal probability distribution. A similar approach, for linear dynamics, is applied in [26] where the conditional Variance-at-Risk (cVaR) is used to minimize constraint violation. In our previous work [10], we ensure safety for a finite set of sampled chance constraints and generalize the associated safety properties using the scenario approach [11].

The *multiplicative* formulation explicitly constrains the product of the marginal probabilities. It was applied in [7] to plan motion under sensing and actuation uncertainty. An alternative marginal formulation is proposed in [19], which bounds the largest marginal constraint violation.

The limitation of marginal approaches is that the bound on the CP of the trajectory is inaccurate, as noted by [21] and [23]. It is shown in [23] that the trajectory CP approaches ∞ and 1, for the additive and multiplicative formulation, respectively, when the number of evaluations in the trajectory increases, regardless of the real CP. Marginal constraints only asses the risk correctly for the first time step and a single obstacle. The risk of the remainder is under- or overestimated. Overestimation of the risk and the associated unsafe space along the time horizon can cause the planning problem to become infeasible and may cause the robot to freeze. Due to these limitations, [21] conditioned marginal chance constraints on being collision free at prior times and evaluated them by truncating the part of the distribution in collision in each time instance. This formulation is more accurate, but is limited to Gaussian distributions.

Joint Chance Constraint

Some authors formulate a *joint* chance constraint on the CP of the planned trajectory. Joint chance constraints estimate the open-loop risk over the time horizon more accurately, making it less likely that the problem becomes infeasible and improving performance. The joint CP can be evaluated by using sampling-based methods [23]. In particular, prior works [22]–[24] consider Importance Sampling Monte-Carlo (ISMC) sampling to *approximate* the CP. An empirical estimate of the constraint violation is obtained by sampling the joint distribution and evaluating the constraint for each sampled *trajectory*. ISMC methods are well suited for estimating risk, but are computationally expensive when planning a trajectory. An alternative is to formulate a mixed-integer problem (e.g., [22]) to decide which samples may be violated, but these problems are hard to solve in real-time.

C. Scenario Optimization

Rather than constraining each of the marginal CPs along the planned trajectory we directly constrain the joint CP of the trajectory. We achieve this through Nonconvex Scenario Optimization (NSO) [11] under an explicit chance constraint on the joint CP. Scenario optimization is well established for convex optimization under uncertainty (see e.g., [27]–[30]) and has recently been extended to the general nonconvex case [11]. NSO is a sampling-based framework for optimization under uncertainty, similar to ISMC, but instead of *averaging* the samples, it *constrains* the solution to the samples, which is computationally more efficient when planning trajectories.

The NSO framework [11] cannot directly be applied to the considered application. The traditional safety assessment needs to repeat the optimization problem once for each scenario and thus cannot provide guarantees in real-time. In addition, unbounded distributions (e.g., Gaussian distributions) cannot be readily incorporated. We address these limitations in this work, significantly reducing the computational complexity of the online safety assessment (Sec. V) and including distributions with unbounded support (Sec. VI). This allows us to apply the framework to online motion planning in real-time.

D. Contribution

The contribution of this paper is three-fold:

1) A novel trajectory optimization method, *Safe Horizon MPC*, that explicitly constrains the collision probability over the full duration of the planned trajectory. This distinguishes our work from previous work, e.g., [4]–[10], where the collision probability is constrained per planning time instance. The idea is to formulate a single collision avoidance constraint over the horizon and to evaluate it by sampling from the distribution of the uncertainty, where each sample represents a trajectory for all obstacles. The more samples are drawn, the higher the probability that the constraint is satisfied. By relying on sampling, our planner is distribution agnostic.
2) An approach that, under a convexity assumption on the iterations of the underlying optimization algorithm, identifies the scenarios that hold the solution in place (known as the *support*) during optimization, in contrast with the framework [11] where the support is computed after optimization. We leverage this information to certify the motion plan online.
3) A method to incorporate *scenario removal* in the NSO framework of [11]. Scenario removal is an extension of the scenario approach where more scenarios are sampled initially such that some of the most limiting scenarios can be removed before optimization. This approach, which has been studied in the convex case (see for example [29], [31]), reduces how conservative the solution of the scenario program is and makes the scenario approach applicable to distributions with unbounded support. As a result, scenario removal makes the online optimization consistent and reduces infeasibility.

We compare our approach in two simulation environments, on a mobile robot and a self-driving vehicle, with pedestrians against marginal baselines and show that our method achieves faster trajectories while maintaining the probabilistic safety specification. In addition, we show that the computation times

scale well with regards to the number of obstacles and the type of distribution. Our planner is implemented in C++/ROS and will be released open source.

E. Notation

Vectors and matrices are expressed in bold and capital bold notation, respectively. We use $\bigwedge$ to denote the "and" operation and $\bigvee$ to denote the "or" operation. The notation $\mathcal{C} \setminus C_i$ refers to the set $\mathcal{C}$ from which the element C_i is removed.

III. PROBLEM FORMULATION

We consider a controlled robot with the general nonlinear discrete-time dynamics

$$\boldsymbol{x}_{k+1} = f(\boldsymbol{x}_k, \boldsymbol{u}_k),$$

where $\boldsymbol{x}_k \in \mathbb{R}^{n_x}$ and $\boldsymbol{u}_k \in \mathbb{R}^{n_u}$ denote the states and inputs, respectively. The robot state is assumed to contain its x-y position $p = [x, y] \in \mathbb{R}^2 \subseteq \mathbb{R}^{n_x}$. Humans in the environment of the robot pose constraints on the navigation envelope. We consider that the motion prediction of the dynamic obstacles is uncertain by modeling the positions of at most M obstacles as random variables. In particular, we denote the uncertain position of Obstacle j at time step k as $\boldsymbol{\delta}_{k,j} \subseteq \boldsymbol{\delta}_k$, where $\boldsymbol{\delta}_k$ contains all obstacle positions at time k. The *joint uncertainty* $\boldsymbol{\delta} = \left[\boldsymbol{\delta}_1^T, \dots, \boldsymbol{\delta}_N^T\right]^T \in \Delta$ stacks the uncertainty over all time steps. Here, Δ denotes the probability space[2] of the joint uncertainty, which is endowed with a σ-algebra $\mathcal{D}$ and a probability measure $\mathbb{P}$.

A. Chance Constrained Planning Problem

The problem setup is visualized in Fig. 2. We model the robot area and obstacle area with n_d discs and a single disc, respectively. The dynamic uncertainty of other road-users affects the navigation envelope of the robot. To constrain the probability of a collision with *any* obstacle along the horizon, we formulate a single chance constraint for collision avoidance. This leads to the following Chance Constrained Problem (CCP)

Problem 1 (CCP)**.**

$$\min_{\boldsymbol{u}\in\mathbb{U},\boldsymbol{x}\in\mathbb{X}} \sum_{k=0}^{N} J(\boldsymbol{x}_k, \boldsymbol{u}_k) \tag{1a}$$

$$\text{s.t.} \quad \boldsymbol{x}_0 = \boldsymbol{x}_{\text{init}} \tag{1b}$$

$$\boldsymbol{x}_{k+1} = f(\boldsymbol{x}_k, \boldsymbol{u}_k),\ k = 0, \dots, N-1 \tag{1c}$$

$$\mathbb{P}\left[\bigwedge_{k=1}^{N}\bigwedge_{d=0}^{n_d}\bigwedge_{j=0}^{M}\left(||\boldsymbol{p}_k^d - \boldsymbol{\delta}_{k,j}||_2 \geq r\right)\right] \geq 1-\epsilon \tag{1d}$$

where $\boldsymbol{x}_{\text{init}}$ denotes the initial state of the robot, objective J is designed to achieve control objectives, $\mathbb{X} \subseteq \mathbb{R}^{Nn_x}$ and $\mathbb{U} \subseteq \mathbb{R}^{Nn_u}$ denote state and input constraints over the horizon, respectively, and (1d) is the collision avoidance chance constraint. Our goal is to compute a control input $\boldsymbol{u}$ and trajectory $\boldsymbol{x}$ under the uncertainty $\boldsymbol{\delta}$ that is collision free with a probability of at least $1-\epsilon$, where ϵ denotes the maximum collision probability over the planned trajectory.
When $\mathbb{P}$ is estimated by a prediction model, the collision avoidance constraint is formulated with respect to an estimate $\hat{\mathbb{P}}$ of $\mathbb{P}$ and guarantees relate to the estimate of the probability distribution.

[2]See e.g., [32] for more details.

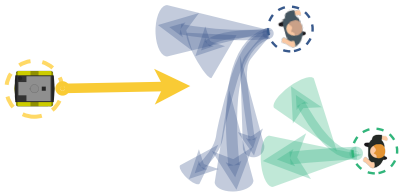


Fig. 2: The chance constrained motion planning problem considered in this paper with the robot (yellow) navigating under probabilistic motion predictions of obstacles (blue/green). The distribution of motion prediction can take any form, but is visualized here with several modes (arrows/shaded regions).

B. Scenario-Based Planning Problem

Directly evaluating chance constraint (1d) is not computationally feasible in closed loop. Our goal is to formulate a sampled deterministic version of the CCP, known as a Scenario Program (SP) [11]. Because (1d) can be enforced when $\boldsymbol{\delta}$ is deterministic, we are able to reformulate the CCP into a problem that can be solved in closed loop. The challenges for safe robot navigation within this framework are to determine the number of samples that must be drawn and, consequently, identify the samples that affect the optimization.

C. Paper Organization

In the following, we first consider a more general CCP formulation that can be solved by the proposed framework. We provide a brief summary of the nonconvex scenario optimization framework [11] to show how this general class of CCPs can be solved via its associated SP. We then present our main results in Section V and Section VI which allows the SP to be solved in closed loop, while maintaining feasibility with respect to the CCP. Section VII shows how to solve Problem 1 by solving its associated SP in an MPC framework. Finally, Sections VIII and IX apply this MPC framework to generate safe motion plans for a robot navigating among pedestrians.

IV. NONCONVEX SCENARIO OPTIMIZATION

In the following, we summarize the main results of the NSO framework [11] that we use to build our motion planning framework. To this end, consider the following generalization of Problem 1:

Problem 2 (General CCP)**.**

$$\min_{\boldsymbol{u}\in\mathbb{U},\boldsymbol{x}\in\mathbb{X}} \sum_{k=0}^{N} J(\boldsymbol{x}_k, \boldsymbol{u}_k) \tag{2a}$$

$$\text{s.t.} \quad \boldsymbol{x}_0 = \boldsymbol{x}_{\text{init}} \tag{2b}$$

$$\boldsymbol{x}_{k+1} = f(\boldsymbol{x}_k, \boldsymbol{u}_k),\ k = 0, \dots, N-1 \tag{2c}$$

$$\mathbb{P}\left[g(\boldsymbol{x}, \boldsymbol{\delta}) \leq 0\right] \geq 1-\epsilon,\ \boldsymbol{\delta} \in \Delta. \tag{2d}$$

The constraints $g(\boldsymbol{x}, \boldsymbol{\delta}) \leq 0$ must be satisfied with a probability of at least $1-\epsilon$. The main idea of scenario optimization [11] is to solve Problem 2 by imposing deterministic constraints for a set of *scenarios* $\boldsymbol{\omega} = \{\boldsymbol{\delta}^{(1)}, \ldots, \boldsymbol{\delta}^{(S)}\} \in \Delta^S$, where each scenario is independently extracted from $\mathbb{P}$ and Δ^S represents the S-fold Cartesian product of Δ associated with drawing S random samples. The number of sampled scenarios is known as the *sample size* S. The SP associated with Problem 2 is given by

Problem 3 (General SP)**.**

$$\min_{\boldsymbol{u}\in\mathbb{U},\boldsymbol{x}\in\mathbb{X}} \quad \sum_{k=0}^{N} J(\boldsymbol{x}_k, \boldsymbol{u}_k) \tag{3a}$$

$$\text{s.t.} \quad \boldsymbol{x}_0 = \boldsymbol{x}_{\text{init}} \tag{3b}$$

$$\boldsymbol{x}_{k+1} = f(\boldsymbol{x}_k, \boldsymbol{u}_k),\ k = 0, \ldots, N-1 \tag{3c}$$

$$g(\boldsymbol{x}, \boldsymbol{\delta}^{(i)}) \leq 0,\ \boldsymbol{\delta}^{(i)} \in \boldsymbol{\omega},\ i = 1, \ldots, S, \tag{3d}$$

where chance constraint (2d) has been replaced by the deterministic constraints for each of the scenarios in (3d).

To simplify notation, we define a *decision* as $\boldsymbol{\theta} := \left[\boldsymbol{x}^T, \boldsymbol{u}^T\right]^T \in \Theta$, where $\Theta = \mathbb{X} \times \mathbb{U}$. Each scenario $\boldsymbol{\delta}^{(i)}$ imposes a constraint $\boldsymbol{\theta} \in \Theta_{\boldsymbol{\delta}^{(i)}} \subset \Theta$ on the decision. Formally, to make a decision based on the scenarios we have a *decision algorithm* $\mathcal{A}$, that maps the scenarios to a decision (i.e., solving Problem 3). This (sub)optimal decision $\boldsymbol{\theta}^* = \mathcal{A}(\boldsymbol{\omega})$ is called the *scenario decision*. The following assumption needs to hold.

Assumption 1. For any finite $S = 1, 2, \ldots$ and for any sample $\boldsymbol{\omega} \in \Delta^S$, it holds that

$$\mathcal{A}(\boldsymbol{\omega}) \in \Theta_{\boldsymbol{\delta}^{(i)}},\ i = 1, \ldots, S. \tag{4}$$

We impose a finite sample size, contrary to [11], to focus on the considered application in which infinite sample sizes are not relevant. Assumption 1 implies that the decision algorithm identifies a feasible solution with respect to the scenario constraints for all possible extractions of samples. We show in Sec. VI how this assumption can be satisfied under unbounded uncertainty. Intuitively, the more scenarios that were used to compute the scenario decision, the lower is the probability that the resulting decision will violate the constraints. Formally, the *violation probability*, $V : \Theta \to [0, 1]$, given by

$$V(\boldsymbol{\theta}) := \mathbb{P}\left[\boldsymbol{\delta} \in \Delta : \boldsymbol{\theta} \notin \Theta_{\boldsymbol{\delta}}\right], \tag{5}$$

defines the probability that a decision $\boldsymbol{\theta}$ violates a newly observed scenario. We also refer to this probability as the *risk* of the decision. Since the decision $\boldsymbol{\theta}$ depends on the realization of the randomly sampled scenarios, the violation probability $V(\boldsymbol{\theta})$ is in itself a random variable over the product probability measure, given by $\mathbb{P}^{\text{S}} = \mathbb{P}\times\ldots\times\mathbb{P}$ (S times). We are therefore interested in lower bounding the *confidence*, which is the *probability* that the scenario decision achieves a risk of at most ϵ. The key variable to obtain this bound is the support subsample, defined as follows.

Definition 1. [11]: Given a multi-sample $\boldsymbol{\omega} = \{\boldsymbol{\delta}^{(1)}, \ldots, \boldsymbol{\delta}^{(S)}\}$, a *support subsample* $\mathcal{C} = \{\boldsymbol{\delta}^{(i_1)}, \ldots, \boldsymbol{\delta}^{(i_n)}\}$ is a tuple of n elements extracted from the multi-sample with $i_1 < i_2 < \ldots < i_n$, which gives the same solution as the original sample, that is,

$$\mathcal{A}(\boldsymbol{\delta}^{(i_1)}, \ldots, \boldsymbol{\delta}^{(i_n)}) = \mathcal{A}(\boldsymbol{\delta}^{(1)}, \ldots, \boldsymbol{\delta}^{(S)}). \tag{6}$$

The cardinality of the support subsample is referred to as the *support size*, that is, $n := |\mathcal{C}|$. In the context of this paper, a scenario is said to be *of support* if it is an element of the considered support subsample. A scenario that can be excluded from a support subsample without changing the solution is said to be *not of support*. For example, a sampled human trajectory $\boldsymbol{\delta}^{(i)}$ can be excluded from the support if it does not change the robot's optimal behavior under the current set of human trajectory samples.

The support size captures the number of scenarios necessary to hold the solution of the SP in place and is strongly correlated with its risk. This correlation can be used to derive a probabilistic guarantee on the solution of the SP using only the support and sample size. Denoting the *confidence* as $1-\beta$, Theorem 1 in [11] provides the following bound

$$\mathbb{P}^{\text{S}}[V(\boldsymbol{\theta}^*) > \epsilon(n)] \leq \sum_{n=0}^{S-1} \binom{S}{n} [1-\epsilon(n)]^{S-n} = \beta. \tag{7}$$

That is, the probability that the scenario decision $\boldsymbol{\theta}^*$ exceeds the acceptable risk ϵ, is upper bounded by β. The function $\epsilon(n) : \{0, \ldots, S\} \to [0, 1]$ is designed subject to (8) and $\epsilon(S) = 1$, which divides the risk over the range of the support from 0 to S. The following mapping devides the risk evenly over all support values

$$\epsilon(n) = \begin{cases} 1, & n > \bar{n}, \\ 1 - \left(\frac{\beta}{S\binom{S}{n}}\right)^{\frac{1}{S-n}}, & n \leq \bar{n}. \end{cases} \tag{8}$$

Notice that the violation probability increases with the support size. The more scenarios that are necessary to support a decision, the higher risk that decision is.

Two limitations prevent the NSO framework from being used in online control. Firstly, to find a support subsample one typically has to solve S optimization problems after the original problem, each time removing one of the scenarios. The set of scenarios for which the solution changes form a support subsample. This is known as the *greedy* algorithm. In closed loop control, solving the problem S times is computationally intractable. We develop a more suitable algorithm in Sec. V. Secondly, Assumption 1 is not satisfied for distributions with unbounded support, such as Gaussian distributions, as outliers can make the problem infeasible. We consider *scenario removal*, similar to the convex case (e.g., [29], [31]) in Sec. VI. Scenario removal ensures feasibility and reduces the variance of the planned trajectory over consecutive iterations.

V. COMPUTING A SAFE SAMPLE SIZE

In the following, we propose an algorithm for estimating the support of an SP during optimization. We then show how this online support estimation can be used to determine if the solution of the scenario program is certifiably safe. In the main application of this paper, our results determine for how many sampled road-user trajectories safety must be guaranteed to

attain a specified risk. We establish this relation by identifying the sampled trajectories that affect the motion plan.

A. Computing the Sample Size Before Optimization

First, let us consider (7) and (8) in a different perspective where the sample size S is computed before optimization for some support $\bar{n}$ that we expect not to exceed. We can formulate the following result which will be useful for online applicability of the proposed method.

Theorem 1. Suppose that Assumption 1 holds true, and set a desired $\epsilon \in [0,1]$, $\beta \in (0,1)$. Given a support limit $\bar{n}$ and a sample size S computed from (7) such that $\epsilon(\bar{n}) \leq \epsilon$, with $\epsilon(n)$ as in (8). Then the solution of Problem 3 computed by decision algorithm $\mathcal{A}$ is feasible for Problem 2 for any $n \leq \bar{n}$.

Proof. Under Assumption 1 and for $\epsilon(n)$ as in (8), [11, Theorem 1] ensures that (7) holds. Since $\epsilon(n)$ in (8) is monotonically increasing in n (i.e., $\epsilon(n+1) > \epsilon(n)$, $\forall n < S$) and given that the computed S ensures that $\epsilon(\bar{n}) \leq \epsilon$, we have that $\epsilon(n) \leq \epsilon, \forall n \leq \bar{n}$. □

This shows that, for a precomputed S, the solution to the SP in Problem 3 can be used to solve the CCP in Problem 2, as long as $n \leq \bar{n}$.

B. Support of Iterations

We now consider how to compute the support n for a nonconvex optimization problem. The support of a convex SP can easily be computed as, under convexity, support constraints are satisfied with equality, i.e., support constraints are active [27], [31]. This property is lost in the nonconvex case. We aim here to reestablish this property by considering, instead of the support of the decision algorithm as a whole, the support of its iterations. With *iteration* we refer to the procedure that is repeated to solve the nonconvex optimization, such that the decision algorithm can be described by a repeated sequence of iterations $\mathcal{A}^l \colon \Theta \times \Delta^S \to \Theta$, $l \in 0, \dots, L$ as

$$\mathcal{A} = \mathcal{A}^L(\dots \mathcal{A}^1(\mathcal{A}^0(\boldsymbol{\theta}^0, \boldsymbol{\omega}), \boldsymbol{\omega}) \dots, \boldsymbol{\omega}). \tag{9}$$

We define the support of an iteration as the set of scenarios that changes its outcome when excluded. To connect the support of the decision algorithm with that of its iterations, consider the following lemma.

Lemma 1. Consider a decision algorithm $\mathcal{A}$, separated according to (9). Its support $\mathcal{C}$ satisfies

$$\mathcal{C} \subseteq \boldsymbol{\omega} \setminus \boldsymbol{\omega}_{\text{ns}}, \quad \boldsymbol{\omega}_{\text{ns}} = \bigcap_{l=0}^{L} \boldsymbol{\omega}_{\text{ns}}^l, \tag{10}$$

where

$$\boldsymbol{\omega}_{\text{ns}}^l = \left\{ \boldsymbol{\delta}^{(i)} \in \boldsymbol{\omega} \mid \mathcal{A}^l(\boldsymbol{\theta}^l, \boldsymbol{\omega}) = \mathcal{A}^l(\boldsymbol{\theta}^l, \boldsymbol{\omega} \backslash \boldsymbol{\delta}^{(i)}) \right\}, \tag{11}$$

is the set of scenarios not of support in iteration l.

Proof. Scenarios in $\boldsymbol{\omega}_{\text{ns}}$ can be excluded for all l without changing the solution, that is, by (10) and (11),

$$\mathcal{A}(\boldsymbol{\theta}^0, \boldsymbol{\omega} \backslash \boldsymbol{\delta}^{(i)}) = \mathcal{A}(\boldsymbol{\theta}^0, \boldsymbol{\omega}) = \boldsymbol{\theta}^*, \tag{12}$$

for all $\boldsymbol{\delta}^{(i)} \in \boldsymbol{\omega}_{\text{ns}}$. Therefore, by Definition 1 all scenarios in $\boldsymbol{\omega}_{\text{ns}}$ are *not* of support for $\mathcal{A}$. The support of $\mathcal{A}$ is therefore in the complement of this set with respect to the set $\boldsymbol{\omega}$ and the result follows. □

The support set obtained through Lemma 1 is an overestimation. It is possible that a scenario changes the solution of an intermediate iteration without changing the final solution.

C. Solving the SP Through Convex Iterations

We now consider the general SP in Problem 3. For this problem, a local optimum can be computed by iteratively linearizing the problem and solving a convex optimization. This makes each iteration a convex scenario optimization[3] for which the support constraints are active [28]. Formally we pose the following two assumptions.

Assumption 2. Each iteration $\mathcal{A}^l$ of decision algorithm $\mathcal{A}$ is convex.

Assumption 3. The solution computed by each iteration $\mathcal{A}^l$ is feasible with respect to its inequality constraints.

The constraints of each iteration are usually linearized scenario constraints (possibly constructed with linearized robot dynamics). It is sufficient to require that these constraints are satisfied since the active linearized scenario constraints still identify the scenarios that affect the solution.

Theorem 2. If iterations $\mathcal{A}^l$ of the decision algorithm satisfy Assumptions 2 and 3, then the support of Problem 3 satisfies

$$\mathcal{C} \subseteq \bigcup_{j=0}^{L} \boldsymbol{\omega}_{\text{active}}^j = \hat{\mathcal{C}}, \quad \hat{n} = |\hat{\mathcal{C}}|, \tag{13}$$

where, with g^l the inequality constraints of $\mathcal{A}^l$,

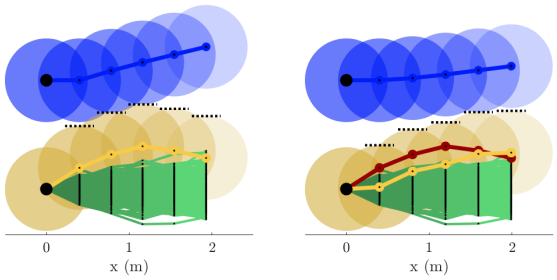
$$\boldsymbol{\omega}_{\text{active}}^l = \{ \boldsymbol{\delta}^{(i)} \in \boldsymbol{\omega} \mid \exists k \; g^l(\boldsymbol{x}_k^l, \boldsymbol{\delta}_k^{(i)}) = 0 \}, \tag{14}$$

denote the active constraints in iteration l.

Proof. Under Assumption 2, the support constraints of iteration l are in the set ω_{active}^l (all constraints are satisfied by Assumption 3). Under convexity, ω_{active}^l is therefore the complement of the set ω_{ns}^l. Invoking Lemma 1 and using De Morgans Law [33], we obtain (13). □

Theorem 2 estimates the support through the aggregated set of active scenarios across all iterations. Assumption 2 (convexity of iterates) and Assumption 3 (feasibility of intermediate iterates) restrict the solvers that can be used with the proposed approach. However, widely available solvers, such as Sequential Quadratic Programming (SQP) [33, Chapter 18], satisfy these criteria.

[3]For example, in Sequential Quadratic Programming (SQP), each iteration $\mathcal{A}^l$ refers to the inner QPs.



(a) No Removal (b) Removed 1

Fig. 3: The optimized robot trajectory (blue) for the SP in Eq. 18 and the sampled obstacle trajectories (green) for $S = 400$. Samples of support (yellow) or that were removed (red) are highlighted.

D. Illustrating Example: SP for 1-D Obstacle Avoidance

In the following, we illustrate the proposed support estimation on a simplified example. We consider a robot following Euler-discretized unicycle dynamics, for $dt = 0.2$s,

$$\begin{aligned} \boldsymbol{p}_{k+1} &= \boldsymbol{p}_k + v_k \begin{bmatrix} \cos(\eta_k) \\ \sin(\eta_k) \end{bmatrix} dt \\ \eta_{k+1} &= \eta_k + \omega_k \, dt. \end{aligned} \tag{15}$$

The environment contains a single obstacle with the same x position as the robot, but uncertain y position,

$$p^{y,\text{obs}}_{k+1} = p^{y,\text{obs}}_k + \zeta_k, \;\; \zeta_k \sim \mathcal{N}(-1.3, 0.07), \tag{16}$$

in which the uncertainty at time k is given by $\delta_k = \zeta_k$. The robot is subject to state constraints $0 \leq v_k \leq 2$ and $-2.0 \leq \omega_k \leq 2.0$. The goal for the robot is to stay above the obstacle along the horizon of $N = 5$ steps, which we enforce with a horizontal collision avoidance constraint, while minimizing its deviation in the y-direction. The sum of robot and obstacle radius is $r = 1.0$m. We formulate the following CCP:

$$\min_{\boldsymbol{u}\in\mathbb{U},\boldsymbol{x}\in\mathbb{X}} \quad \sum_{k=1}^{N} (p^y_k)^2 + \omega_k^2 + (v_k - 2.0)^2 \tag{17a}$$

$$\text{s.t.} \quad \boldsymbol{x}_{k+1} = f(\boldsymbol{x}_k, \boldsymbol{u}_k), \; \forall k, \tag{17b}$$

$$\mathbb{P}\left[\bigwedge_{k=1}^{N} (p^y_k \geq \delta_k + r)\right] \geq 1 - \epsilon, \;\; \boldsymbol{\delta} \in \Delta. \tag{17c}$$

The associated SP (see Fig. 3a) is given by

$$\min_{\boldsymbol{u}\in\mathbb{U},\boldsymbol{x}\in\mathbb{X}} \quad \sum_{k=1}^{N} (p^y_k)^2 + \omega_k^2 + (v_k - 2.0)^2 \tag{18a}$$

$$\text{s.t.} \quad \boldsymbol{x}_{k+1} = f(\boldsymbol{x}_k, \boldsymbol{u}_k), \; \forall k, \tag{18b}$$

$$p^y_k \geq \delta^{(i)}_k + r, \;\; \forall k, \;\; \boldsymbol{\delta}^{(i)} \in \boldsymbol{\omega}, \;\; i = 1, \ldots, S, \tag{18c}$$

which is nonconvex through the equality constraints (18b). We do not consider the risk ϵ achieved by (18) in this example. Instead, we compare greedy support estimation [11] with ours (Eq. 13) by solving 25 realizations of SP (18) using SQP for each of the sampling sizes $S = \{100, 200, \ldots, 1000\}$. The proposed support estimate is larger than the greedy support estimate in $8/250$ experiments and smaller in 1 experiment (i.e., it is slightly more conservative)[4]. However, the computation times, as visualized in Fig. 4 indicate that the proposed method is faster and scales more favorably. For example, with 1000 samples it requires approximately 13.2% of the computation time, improving further as the sample size increases.

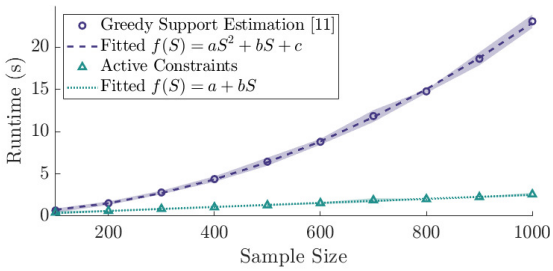


Fig. 4: Mean (markers) and standard deviation (shaded region) of support estimation computation times.

E. The Support Limit and Associated Guarantees

We recall that, through Theorem 1, the risk of the planned trajectory is certified by the support of the decision algorithm with which Problem 3 is solved and for which Theorem 2 provides an estimate. If we solve Problem 3 and find that $\hat{n} > \bar{n}$, then Theorem 1 does not provide guarantees at the specified ϵ. There are two reasons why this could happen. Either the CCP is infeasible for the desired ϵ (including conservatism of the scenario approach) or there may exist a safe solution under the specified ϵ, but under the applied values of $\bar{n}$ and the associated S the SP did not find it.

In the first case, nothing can be done to provide a safe solution. The second case is more problematic because a feasible solution exists. Several methods can be used to deal with this case, for example, running the SP for increasing sample sizes (similarly to [34]) or running several SPs in parallel (similarly to [35]). We consider these options future work and apply in this paper a constant value of $\bar{n}$ (and S) that is high enough that in practice Theorem 1 holds for almost all iterations in our experiments. This approach can be conservative, as in many iterations the support may be smaller than our bound.

We leverage two ideas to keep the support (and sample size) of the SP small. First, we formulate scenario constraints (see Sec. VIII) such that other scenarios may become *redundant*, meaning that the optimization can be described without them. Redundant samples cannot be of support and therefore reduce the complexity of the problem. For example, when sampling human trajectories, only the most extreme cases affect the motion plan. Appendix A provides mathematical and empirical insights for this idea. Second, we use the support estimate from Theorem 2 to evaluate during optimization if the optimization problem should continue iterating, or should be stopped to provide a suboptimal, but safe solution. The support estimate in iteration l is given by

$$\mathcal{C}^l \subseteq \bigcup_{j=0}^{l} \boldsymbol{\omega}^j_{\text{active}} = \hat{\mathcal{C}}^l, \quad \hat{n}^l = |\hat{\mathcal{C}}^l|. \tag{19}$$

[4]We note that both support estimations are not necessarily minimal.

Theorem 3. Suppose that that in iteration l of $\mathcal{A}$, $\hat{n}^l > \bar{n}$. If, in at least one previous iteration $m < l$, all constraints of Problem 3 were satisfied and $\hat{n}^m \leq \bar{n}$. Then the solution θ^m is a feasible solution with support $n^m \leq \bar{n}$.

Proof. The solution at iteration m is feasible given that all constraints of Problem 3 are satisfied. Terminating the optimization in principle adds to the support since it influences the outcome of the decision algorithm. However, since the decision algorithm with termination is equivalent to a decision algorithm with constant iteration length (with $L = m$), this support does not need to be included. Hence, the support is estimated by $\hat{n}^m$ and by Theorem 2, $\hat{n}^m \geq n^m$, such that $n^m \leq \bar{n}$. □

Because Theorem 3 needs the iterates to be feasible before the support limit is exceeded, we cannot ensure via termination that $n \leq \bar{n}$. In practice, termination may reduce the support.

F. Summary

We modeled the planning problem with a constraint on the joint probability of collision (Problem 1) as a specific case of Problem 2. This more general CCP can be solved via the SP in Problem 3 as long as the sample size (e.g., number of sampled road-user trajectories) is sufficiently large. Theorem 1 showed that for a constant sample size, the motion plan can be certified by the support (i.e., number of samples that influence the plan). When iterations of this algorithm are convex, Theorem 2 provides a cheap support estimate that is computed during optimization. This support estimate, through Theorem 3, could be used to reduce the support of the optimization. Through this technique Problem 1 can be solved efficiently.

VI. Scenario Removal

So far we have shown under what conditions the solution to the SP in Eq. 3 solves the CCP in Eq. 2 if Assumption 1 is satisfied. In this section, we consider how Assumption 1 can be satisfied, particularly for distributions with unbounded support. The SP in Eq. 3 does not satisfy this assumption outright. For example, in the motion planning application there is always a non-zero probability that a sampled pedestrian overlaps with the robot, making the SP infeasible even when the CCP admits a feasible solution. We introduce in this section a modified SP with *scenario removal* [31] to satisfy Assumption 1 under unbounded support. Scenario removal allows the SP to remove limiting samples (potential outliers) after sampling, at the cost of drawing more samples initially.

Scenario removal additionally reduces the variance of the solution to the SP. This makes the solution more consistent over consecutive iterations. In a motion planning context, scenario removal makes it possible to ignore extremely unlikely road-user trajectories that lead to too conservative motion plans under the specified risk.

A. Scenario Removal SP

Consider a *removal algorithm* $\mathcal{R}$ that identifies scenarios to be removed from the SP in Problem 3 with the goal to reduce the cost or to make the problem feasible. That is, a function $\mathcal{R} : \Theta \times \Delta^S \to \Delta^r$,

$$\mathcal{R}(\boldsymbol{\theta}, \boldsymbol{\omega}) = (\boldsymbol{\delta}^{(i_1)}, \dots, \boldsymbol{\delta}^{(i_r)}). \tag{20}$$

The intent is that constraints indexed by the removal function are *removed* from the scenario constraints in Problem 3 when solving the SP (Eq. 3). With regards to Assumption 1 we pose the following assumption on $\mathcal{R}$.

Assumption 4. The removal algorithm removes any scenarios that prevent a feasible solution. Formally, Problem 3, where (3d) excludes removed scenarios as

$$g(\boldsymbol{x}, \boldsymbol{\delta}^{(i)}) \leq 0, \boldsymbol{\delta}^{(i)} \in \boldsymbol{\omega} \backslash \mathcal{R}(\boldsymbol{\theta}, \boldsymbol{\omega}), \tag{21}$$

is assumed to be feasible.

We now show how the SP in Problem 3 can be modified under scenario removal.

Theorem 4. The SP in Problem 3, including a scenario removal algorithm $\mathcal{R}$ satisfying Assumption 4 is a decision algorithm that satisfies Assumption 1.

Proof. Inspired by [31, Chapter 5], we may equivalently represent the SP with removal by extending the decision variables with S Boolean variables $\boldsymbol{\xi} = \left[\xi^{(1)}, \dots, \xi^{(S)}\right]$, $\boldsymbol{\xi} \in \{0, 1\}^S$, such that $\boldsymbol{\theta} = (\boldsymbol{x}, \boldsymbol{u}, \boldsymbol{\xi})$ and where each boolean variable is 1 if and only if the scenario it indexes is contained in $\mathcal{R}$. We then reformulate the original SP in Problem 3 as

Problem 4 (SP with Removal)**.**

$$\min_{(\boldsymbol{x}, \boldsymbol{y}, \boldsymbol{\xi}) \in \Theta} \quad \sum_{k=0}^{N} J(\boldsymbol{x}_k, \boldsymbol{u}_k) \tag{22a}$$

$$\text{s.t.} \quad \boldsymbol{x}_0 = \boldsymbol{x}_{\text{init}} \tag{22b}$$

$$\boldsymbol{x}_{k+1} = f(\boldsymbol{x}_k, \boldsymbol{u}_k, \boldsymbol{\delta}_k^{(i)}) \tag{22c}$$

$$g(\boldsymbol{x}, \boldsymbol{\delta}^{(i)}) \leq \xi^{(i)} M, \ \boldsymbol{\delta}^{(i)} \in \boldsymbol{\omega}, \forall i, \tag{22d}$$

The key difference is that the constraint formulation $g(\boldsymbol{x}, \boldsymbol{\delta}^{(i)}) \leq 0$ is modified to $g(\boldsymbol{x}, \boldsymbol{\delta}^{(i)}) \leq \xi^{(i)} M$, where a big M formulation ($M \gg 0$) removes the scenario constraint when $\xi^{(i)} = 1$.

To show that Problem 4 satisfies Assumption 1 (i.e., it admits a feasible solution for all possible sample extractions $\boldsymbol{\omega}$), we note that under the modified formulation $\boldsymbol{\theta} \in \Theta_{\delta^{(i)}}$ can hold in two ways: the original scenario constraint is satisfied (i.e., $g(\boldsymbol{x}, \boldsymbol{\delta}^{(i)}) \leq 0$) or the scenario is removed (i.e., $\xi^{(i)} = 1$). Under Assumption 4, a feasible solution to Problem 4 exists for all scenarios where $\xi^{(i)} = 0$, while scenarios where $\xi^{(i)} = 1$ are removed. Therefore, for any sample extraction $\boldsymbol{\omega}$, $\mathcal{A}(\boldsymbol{\omega}) \in \Theta_{\delta^{(i)}}, \forall i = 1, \dots, S$. □

Because the removal of constraints may increase the risk[5], the sample size of the SP must be increased to guarantee that the risk remains the same while some scenarios are removed after sampling. We derive the following result.

[5]Note that removal of constraints is different than excluding constraints that are not of support. Removing constraints changes the solution, adds support and increases the risk.

Theorem 5. The support of Problem 4 includes all removed scenarios, that is,

$$\mathcal{R}(\boldsymbol{\theta}, \boldsymbol{\omega}) \subseteq \mathcal{C}. \tag{23}$$

Proof. Scenarios removed by $\mathcal{R}$ hold the solution over the variables $\boldsymbol{\xi}$ in place, since excluding a removed scenario i from $\boldsymbol{\omega}$, would lead to $\xi^{(i)} = 0$ instead of $\xi^{(i)} = 1$. Thus, by definition, all removed scenarios must be included in the support of Problem 4. □

Although $\boldsymbol{\xi}$ are incorporated in the optimization problem in Theorem 4, the scenarios to be removed can be determined before or during optimization through $\mathcal{R}$ such that no mixed integer optimization needs to be solved. Rather, we solve the original SP in Problem 3 without the scenarios indexed by $\mathcal{R}$ and Theorem 5 shows how Theorem 1 can be applied to guarantee probabilistic safety.

B. Illustrating Example Continued

To illustrate that scenario removal improves consistency of the SP, we continue the illustrating example. Fig. 3b depicts a solution to the example where one scenarios is removed, which decreases the trajectory cost compared to Fig. 3a. We perform a quantitative analysis on scenario removal, solving the SP in Eq. 18 for $R \in \{0, 2, \dots, 20\}$ while updating the sample size according to Theorem 5 with $\epsilon = 0.1, \beta = 10^{-6}$, resulting in $S \in \{290, 390, \dots, 1250\}$. We solve the problem 100 times for each R. Fig. 5 visualizes the mean and variance of the resulting cost and the empirical risk, validated by monte-carlo sampling with 10^4 samples.
Because outliers are removed, the cost decreases in value and variance for any non zero removal size. For larger removal sizes, the variance of the cost decreases mildly with diminishing returns. The risk is consistently less conservative with removal.

VII. SAFE HORIZON MODEL PREDICTIVE CONTROL

So far, we have posed the robot navigation problem as a CCP under joint chance constraints (Problem 1), a specific case of Problem 2, that we can solve via an SP (Problem 4). In Sec. V, we showed how the safety of the solution computed by the SP can be assessed in online control. In this section, we describe how the proposed framework can be applied in closed loop for safe robot navigation.

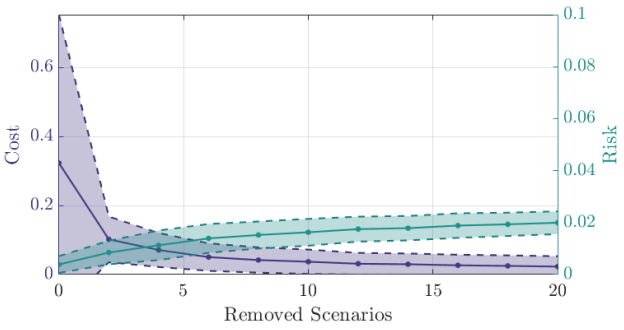


Fig. 5: Cost (purple) and empirical risk (green) distributions for the illustrating example in Section VI-B. The sample size for each removal size is computed using Theorem 4. Colored intervals denote the variance over 100 repeated experiments.

Algorithm 1: Safe Horizon MPC

1 **Input:** ϵ, β, $\bar{n}$, R and $\mathbb{P}$
2 $S \leftarrow$ Bisection of Eq. 8 using $\epsilon, \beta, \bar{n}$
3 **while** True **do**
4 $\quad \boldsymbol{\omega} \leftarrow$ Sample $\mathbb{P}$
5 $\quad$ **for** $l = 1, \dots, L$ **do**
6 $\quad\quad \boldsymbol{\theta}^l \leftarrow$ Solve an iteration of the SP (Eq. 3)
7 $\quad\quad \boldsymbol{\omega}^l_{\text{active}} \leftarrow$ Determine active scenarios (Eq. 14)
8 $\quad\quad \hat{\mathcal{C}}^l, \hat{n}^l \leftarrow$ Aggregate the support (Eq. 19)
9 $\quad\quad$ **if** $\hat{n}^l > \bar{n}$ and Eqs. (3c) and (3d) satisfied at $l = m$ **then**
10 $\quad\quad\quad \boldsymbol{\theta}^* = \boldsymbol{\theta}^m$
11 $\quad\quad\quad$ **break**
12 $\quad\quad \boldsymbol{\xi}^l \leftarrow$ Remove scenarios (Eq. 20)
13 $\quad$ Actuate $\boldsymbol{u}_0^*$

A. MPC Algorithm

We formulate and solve the Model Predictive Control (MPC) problem in Algorithm 1. Before deploying the controller, we compute the sample size from the desired risk, confidence and support limit (line 2). For completeness, we provide with this publication a Jupyter notebook [36] that performs this computation using a bisection of (8). In each control iteration, we sample the modeled distribution of the uncertainty to obtain a set of samples (line 4). Then, we repeatedly solve Problem 3 with the samples and aggregate the support estimate by computing the active constraints (line 6-8). If the support estimate exceeds the support limit and there was a previously safe solution, we return that solution (line 9-11). If there was no previous solution, we slow down the robot. The first input is sent to the system (line 13).

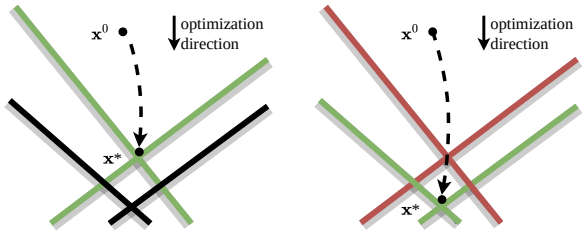
B. Removing Scenarios

In line 12 we remove scenarios to improve the solution and ensure feasibility as detailed in Sec. VI. Function $\mathcal{R}$ updates the removed scenarios in each iteration[6]. Removing the active constraints is an effective way to reduce the cost. These scenarios restrict the optimization, are already of support and are available without additional computations. They represent for example the most restrictive sampled human trajectories in our main application. We therefore define the removal strategy for SH-MPC as

$$\mathcal{R}(\boldsymbol{\theta}, \boldsymbol{\omega}) = \bigcup_{j=0}^{l} \left(\boldsymbol{\omega}^j_{\text{active}} \cup \boldsymbol{\omega}^j_{\text{infeasible}} \right), \tag{24}$$

where $\boldsymbol{\omega}^j_{\text{infeasible}}$ denote the scenarios that must be removed to satisfy Assumption 4. Active constraints $\boldsymbol{\omega}^j_{\text{active}}$ can be

[6] $\mathcal{R}$ must be carefully selected to reduce the cost. If it does not, then the added support quantified in Theorem 5 results in conservative solutions.



(a) No scenarios are removed, the support is 2.

(b) After removing 2 scenarios (linear constraints), the cost decreases but all 4 scenarios are in the support set.

Fig. 6: An illustration of scenario optimization without (left) and with (right) scenario removal. The dots denotes the initial guess and solution to the optimization. The thick lines depict inactive (black), active (green) and removed (red) constraints.

removed up to some maximum of R scenarios. Accordingly, the support increases by at least R through Theorem 5.
Even when removed scenarios are already of support, as is the case in (24), the support of the optimization still increases. Removing scenarios causes underlying scenarios to become of support as illustrated in Fig. 6. Removing 2 scenarios in Fig. 6a leads to Fig. 6b. The cost decreases, but all 4 scenarios are of support, making its solution higher risk.

VIII. APPLICATION: SAFE MOTION PLANNING AROUND PEDESTRIANS

In this section, we apply Safe Horizon MPC as derived in Sec. VII to compute probabilistic safe trajectories in real-time for a mobile robot navigating around pedestrians.

A. Collision Avoidance Scenario Program

In the following, we let the joint uncertainty in Problem 1 represent the future motion of all humans near the robot, that is, $\Delta = \mathbb{R}^{2MN}$, which represents the x-y positions of M obstacles over N time steps. Based on obstacle tracking information, a perception module provides the joint probability distribution of obstacle trajectories $\mathbb{P}$. This distribution is sampled to obtain the scenarios that describe positions of all obstacles along the planning horizon. With these scenarios, $\boldsymbol{\delta}^{(i)} \in \boldsymbol{\omega}$, we can construct the SP for Problem 1 as follows:

$$\min_{\boldsymbol{u}\in\mathbb{U},\boldsymbol{x}\in\mathbb{X}} \quad \sum_{k=0}^{N} J(\boldsymbol{x}_k, \boldsymbol{u}_k) \tag{25a}$$

$$\text{s.t.} \quad \boldsymbol{x}_0 = \boldsymbol{x}_{\text{init}} \tag{25b}$$

$$\boldsymbol{x}_{k+1} = f(\boldsymbol{x}_k, \boldsymbol{u}_k),\ k = 0, \dots, N-1 \tag{25c}$$

$$\bigwedge_{i=0}^{S} \bigwedge_{k=1}^{N} \bigwedge_{d=0}^{n_d} \bigwedge_{j=0}^{M} \left(||\boldsymbol{p}_k^d - \boldsymbol{\delta}_{k,j}^{(i)}||_2 \geq r \right), \tag{25d}$$

where, for brevity, the scenario constraints are bundled using the "and" operator. The collision-free space described by (25d) is nonconvex (see Fig. 7a) and can lead to high support. To reduce the support and computation time of the problem, we first linearize the collision regions with respect to the previously planned robot trajectory (denoted $\hat{\boldsymbol{p}}$). After linearization, each scenario is associated with a linear constraint (depicted as lines in Fig. 7b). Details on how this reduces the support are included in Appendix A. For a previous robot position $\hat{\boldsymbol{p}}_k$ and obstacle position $\boldsymbol{\delta}_k$ the constraints are given by

$$\mathcal{H}(\hat{\boldsymbol{p}}_k, \boldsymbol{\delta}_k) = \{\boldsymbol{p}_k \mid \boldsymbol{A}(\hat{\boldsymbol{p}}_k, \boldsymbol{\delta}_k)^T \boldsymbol{p}_k \leq b(\hat{\boldsymbol{p}}_k, \boldsymbol{\delta}_k)\}, \tag{26}$$

where

$$\boldsymbol{A}(\hat{\boldsymbol{p}}_k, \boldsymbol{\delta}_k) = \frac{\boldsymbol{\delta}_k - \hat{\boldsymbol{p}}_k}{||\boldsymbol{\delta}_k - \hat{\boldsymbol{p}}_k||},\ b(\hat{\boldsymbol{p}}_k, \boldsymbol{\delta}_k) = \boldsymbol{A}^T \boldsymbol{\delta}_k - r. \tag{27}$$

The linearized collision region contains the original collision region, preserving the probabilistic guarantees. In addition, the linearization is applied locally to each timestep k and robot disc d, resulting in a locally accurate approximation of the original collision regions. For the linearized constraints, we obtain the following SP:

Problem 5 (SP)**.**

$$\min_{\boldsymbol{u}\in\mathbb{U},\boldsymbol{x}\in\mathbb{X}} \quad \sum_{k=0}^{N} J(\boldsymbol{x}_k, \boldsymbol{u}_k) \tag{28a}$$

$$\text{s.t.} \quad \boldsymbol{x}_0 = \boldsymbol{x}_{\text{init}} \tag{28b}$$

$$\boldsymbol{x}_{k+1} = f(\boldsymbol{x}_k, \boldsymbol{u}_k),\ k = 0, \dots, N-1 \tag{28c}$$

$$\bigwedge_{i=0}^{S} \bigwedge_{k=1}^{N} \bigwedge_{d=0}^{n_d} \bigwedge_{j=0}^{M} \left(\boldsymbol{p}_k^d \in \mathcal{H}\left(\hat{\boldsymbol{p}}_k^d, \boldsymbol{\delta}_{k,j}^{(i)}\right)\right). \tag{28d}$$

B. Improved Computational Efficiency

To further reduce the computational demand of this formulation, consider that Constraints (28d) can be reordered as

$$\bigwedge_{k=1}^{N} \bigwedge_{d=0}^{n_d} \left[\bigwedge_{i=0}^{S} \bigwedge_{j=0}^{M} \left(\boldsymbol{p}_k^d \in \mathcal{H}\left(\hat{\boldsymbol{p}}_k^d, \boldsymbol{\delta}_{k,j}^{(i)}\right)\right)\right], \tag{29}$$

to pair constraints that apply to a single robot disc position $\boldsymbol{p}_k^d$. Because of the overlap between the constraints, each of these constraint pairings can be described by a small subset of the constraints (see Fig. 7b). Thus, the constraints (28d) can be reduced to a free-space polytope before optimization, which significantly reduces computation times. We denote the polytope for robot disc d and stage k as

$$\mathcal{P}_k^d = \left\{ \boldsymbol{p}_k^d \;\middle|\; \bigwedge_{c=0}^{n_{\mathcal{H}}} \boldsymbol{p}_k^d \in \mathcal{H}_c \right\}. \tag{30}$$

By constructing this polytope, the number of constraints is reduced from $10^3 - 10^4$ to no more than $n_{\mathcal{H}} = 20$ constraints. In addition, the computations necessary to construct the polytope are cheap in 2D. Our algorithm uses a recursive search that typically performs this computation in less than 100 μs. The final SP that is solved online is given by

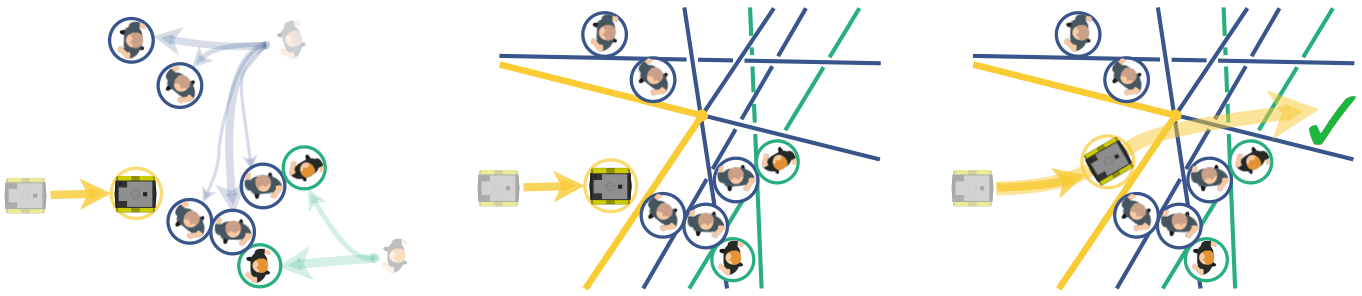
$$\min_{\boldsymbol{u}\in\mathbb{U},\boldsymbol{x}\in\mathbb{X}} \quad \sum_{k=0}^{N} J(\boldsymbol{x}_k, \boldsymbol{u}_k) \tag{31a}$$

$$\text{s.t.} \quad \boldsymbol{x}_0 = \boldsymbol{x}_{\text{init}} \tag{31b}$$

$$\boldsymbol{x}_{k+1} = f(\boldsymbol{x}_k, \boldsymbol{u}_k),\ k = 0, \dots, N-1 \tag{31c}$$

$$\boldsymbol{p}_k^d \in \mathcal{P}_k^d, \quad \forall k, \forall d. \tag{31d}$$

Its solution is a safe trajectory as visualized in Fig. 7c.



(a) Scenarios are sampled from the trajectory distributions. Each time instance of SP (Eq. 25) is associated with a set of sampled obstacle positions as visualized by the green and blue circled pedestrians.

(b) Linear constraints are constructed between sampled obstacles and the robot, and are reduced to a probabilistic safe polytope for each time instance and robot disc.

(c) SP (Eq. 31) is solved via Algorithm 1. The resulting trajectory is certified up to a probabilistic bound.

Fig. 7: Schematic illustration of Safe Horizon MPC applied to a mobile robot.

C. Implementation Details

We apply our framework in a Model Predictive Contouring Control formulation [15], which tracks a reference path and reference velocity while penalizing robot inputs[7]. We solve the SP in Eq. 31 using the Forces Pro SQP solver [37] with at most 15 iterations. We reduce the variance of the solution by removing $R = 1$ scenarios through $\mathcal{R}$ as defined in Eq. (24). Including the support of removal we set $\bar{n} = 9$ for which Theorem 1 almost always certifies the solution (if not, we slow down the robot). For a risk of $\epsilon = 0.05$ and confidence of $\beta = 0.01$ the required sample size is $S = 1237$. The linearization of the constraints requires the previous plan to be feasible. We make use of a projection step to ensure that this holds in all time steps. This projection step consists of a projection orthogonal to the direction of robot movement, which almost always results in a feasible plan. In the remaining cases, we solve a feasibility program using Douglas-Rachford Splitting [38]. Our motion planner is implemented in C++/ROS and will be released open source.

IX. SIMULATION RESULTS

This section compares our SH-MPC with baseline methods that constrain the marginal CP, that is, the independent CP per time instance and obstacle. In the first set of simulations, we consider a robot moving through an environment with pedestrians (see Fig. 9) in which the robot is modeled by a kinematic unicycle model [39]. We assume throughout that the distribution of pedestrian motion is known, in order to evaluate the performance of the planner in isolation (i.e., without prediction errors). We first validate on a Gaussian case, where the baselines may leverage the shape of the distribution to accurately approximate the probabilistic collision-free space. A video of the simulations accompanies this paper [40].

A. Baselines

The first baseline [4] (referred to as "Gaussian") is strictly applicable to Gaussian distributions. It approximates the collision probability via the Cumulative Density Function (CDF) of the Gaussian distribution. The linearized collision avoidance chance constraint

$$\mathbb{P}\left[\boldsymbol{a}_{k,j}^T(\boldsymbol{p}_k - \boldsymbol{\delta}_{k,j}) \geq r\right] \geq 1-\epsilon_k,\ \boldsymbol{a}_{k,j} = \frac{\boldsymbol{p}_k - \boldsymbol{\delta}_{k,j}}{||\boldsymbol{p}_k - \boldsymbol{\delta}_{k,j}||}, \quad (32)$$

is equivalent, under a Gaussian distribution of $\boldsymbol{\delta}_{k,j}$, to

$$\boldsymbol{a}_{k,j}^T(\boldsymbol{p}_k - \boldsymbol{\delta}_{k,j}) - r \geq \text{erf}^{-1}(1-2\epsilon_k)\sqrt{2\boldsymbol{a}_{k,j}^T\boldsymbol{\Sigma}\boldsymbol{a}_{k,j}}, \quad (33)$$

where erf^{-1} is the inverse standard error function and $\boldsymbol{\Sigma}$ is the covariance matrix of the uncertainty. This constraint is imposed separately for each time step and obstacle.

The second baseline S-MPCC [10] is generally applicable. It solves an SP where scenario constraints are posed on the marginal distributions. For each time k, it samples from the independent obstacle distribution at k to obtain a collision-free polygon. It is assumed that all constraints in the polygon are of support (set to 20 in these simulations), which for $\epsilon_k = 0.0025$ requires $S = 75946$. Samples in the center of the distribution are pruned in the Gaussian case to reduce the number of samples considered online.

Since SH-MPC is characterized by a single bound ϵ on the trajectory CP, while the baselines specify bounds ϵ_k on the CP for each k and for each obstacle, we consider three versions of the baselines. The first sets $\epsilon_k = \epsilon$, which is not provably safe, but relies on updates of the controller to remain safe. The second version sets $\epsilon_k = \frac{\epsilon}{N}$, accounting for the marginal approximation over time since $\sum_k \epsilon_k = \epsilon$, but ignoring marginalization per obstacle. The third version accounts for both marginalizations, setting $\epsilon_k = \frac{\epsilon}{NM}$ such that $M\sum_k \epsilon_k = \epsilon$. Only this last version attains the same safety guarantee as SH-MPC. We consider this version in crowded environments, where the marginal CP is violated otherwise.

B. Weights and Parameters

In the following simulations, the planners are only differentiated by their collision avoidance constraints. We use the same solver, weights and cost function for all methods. The weights of the MPC problem are given in Table II. To motivate overtaking behavior, the MPC is tuned differently for the two considered distributions. Aside from the weights, we define

[7] In contrast to [15], our objective does not use repulsive forces from the obstacles.

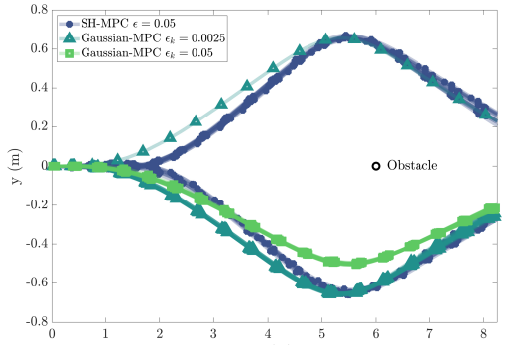


Fig. 8: Trajectories around a static obstacle where motion predictions follow a Gaussian distribution.

a horizon of $N = 20$ steps, with a discretization step of 0.2 s, giving a time horizon of 4.0 s. The control rate is 20 Hz corresponding to a sampling time of 50 ms. The computer running the simulations is equipped with an Intel i9 CPU@2.4GHz.

TABLE II: Weights of the MPC problem.

Simulation (Sec.)	Velocity	Ang. Velocity	Contour	Lag
Trajectories (IX-C)	0.05	0.05	0.02	0.1
Gaussian (IX-D)	0.05	0.05	0.001	0.1
GMM (IX-E)	0.15	0.05	0.005	0.1

C. Static Obstacle Trajectory Comparison

We first compare trajectories of SH-MPC and the Gaussian baseline in an environment with one static obstacle where the motion predictions follow a Gaussian distribution. The pedestrian dynamics are given by

$$\boldsymbol{\delta}_{k+1} = \boldsymbol{\delta}_k + \boldsymbol{\delta}_{w,k} dt, \quad \boldsymbol{\delta}_{w,k} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}_w), \tag{34}$$

where $\boldsymbol{\Sigma}_w \in \mathbb{R}^{2\times 2}$ is a diagonal covariance matrix (i.e., random variables in the x and y direction are independent). Its diagonal entries $\sigma_{wx} = 0.3, \sigma_{wy} = 0.3$ are kept constant over the horizon. For the marginal Gaussian baselines, the distribution of the uncertainty in future steps is propagated using

$$\sigma^2_{x,k+1} = \sigma^2_{x,k} + \sigma^2_{wx}, \ \sigma^2_{y,k+1} = \sigma^2_{y,k} + \sigma^2_{wy}, \tag{35}$$

while for SH-MPC we specify directly the dynamics in Eq. (34). We bound the CP for SH-MPC at $\epsilon = 0.05$ and the marginal CP at $\epsilon_k = 0.05$ and $\epsilon_k = 0.0025$ (5% over $N = 20$ steps) for the baselines. The obstacle is positioned at $(6.0, 0.0)$ and does not move in the simulation. The pedestrian and robot radius are 0.3 m and 0.325 m respectively. We tune the controller to follow the reference trajectory on the x-axis as close as possible and use the same weights for all methods (see Table II). The resulting trajectories over 25 repetitions are depicted in Fig. 8.

The trajectories indicate that the Gaussian baselines need to react faster to the obstacle. SH-MPC and the $\epsilon_k = 0.0025$ Gaussian baseline pass at a similar distance to the obstacle, but under SH-MPC the robot can steer later, possibly leading to faster trajectories. The Gaussian baseline at $\epsilon_k = 0.05$ is faster, but does not meet the safety specification, as we will show in the following simulations.

D. Mobile Robot Simulations - Gaussian

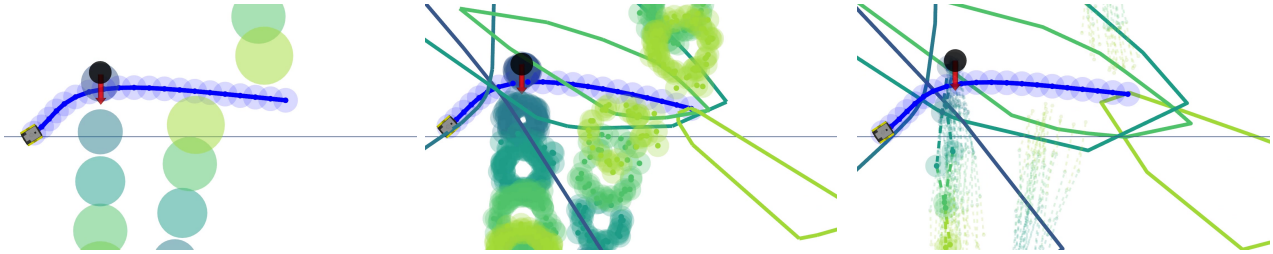
We consider an environment with multiple dynamic pedestrians following dynamics similar to the previous case,

$$\boldsymbol{\delta}_{k+1} = \boldsymbol{\delta}_k + (\boldsymbol{v} + \boldsymbol{\delta}_{w,k})dt, \quad \boldsymbol{\delta}_{w,k} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}_w), \tag{36}$$

but where $\boldsymbol{v} \in \mathbb{R}^2$ describes a constant velocity. We validate the actual CP of the motion plan offline after the experiments for all methods through Monte Carlo sampling in which the dynamics in Eq. (36) are used to generate the samples. The CP is computed by dividing the number of samples where the robot and obstacle discs overlap at any stage by the total number of samples (set to 10^5). The marginal CP (CP_k) is computed without taking prior collisions into account. We validate in a scenario with 4 and 8 pedestrians, respectively. Fig. 9 depicts snapshots of the simulations with 4 pedestrians. The results with 4 pedestrians are summarized in Table III. The Gaussian method bounds the marginal CP (CP_k) accurately. However when $\epsilon_k = 0.05$, the CP of the plan is 0.1271 which exceeds 0.05. The trajectories are safe under $\epsilon_k = 0.0025$, but are conservative with a maximum overall CP of 0.0069. In contrast, SH-MPC attains a higher, but still safe maximum CP of 0.0113. We do not observe a significant difference in the performance metrics of this simulation, the Gaussian method at 0.0025 is slightly faster than SH-MPC but travels more distance. The repeated computations of the controller are likely responsible for the observed similarity in performance. Because marginal methods are accurate over the short term, they become less conservative close to obstacles. It is worth noting that while the Gaussian method uses information of the distribution, SH-MPC and S-MPCC are only using samples of the distribution. S-MPCC is more conservative than the other two methods as a result of its conservative support estimate. For the case of 8 pedestrians, results are summarized in Table IV. In these simulations, the marginal CP of the Gaussian method exceeds the specification. Fig. 12 illustrates on a 1-dimensional example that this is a result of the marginalization per obstacle. In this simulation we therefore also compare against the Gaussian baseline at $\epsilon_k = \frac{0.0025}{8} = 0.0003125$, which attains the same safety guarantees as SH-MPC. The results indicate that SH-MPC is able to move through this environment significantly faster than the baseline with the same safety guarantees. Our method shows no significant change in its overall CP compared to the previous simulation as it considers the joint distribution, while the risk of the baselines has more than doubled. Additionally, the safe baseline with $\epsilon_k = 0.0003125$ is excessively conservative with a CP of 0.0016 versus 0.0128 of SH-MPC.

E. Mobile Robot Simulations - Gaussian Mixture Model

We now modify the distribution to incorporate a probability that the pedestrians will cross. We encode this scenario with a Markov Chain (see Fig. 10) that changes pedestrian movement from horizontal to diagonal in addition to the Gaussian process



(a) Gaussian baseline at $\epsilon_k = 0.0025$. (b) S-MPCC at $\epsilon_k = 0.0025$. (c) SH-MPC at $\epsilon = 0.05$.

Fig. 9: Snapshot of the simulation under Gaussian pedestrian motion. Pedestrians are depicted by black circles with direction of motion highlighted in red. The robot plan and its collision area is drawn in blue. For all methods visualizations are shown in blue to green colors for stages $0, 5, 10, 15$ and 19, respectively. (a) Circles show the level sets of the Gaussian distribution at the specified ϵ. (b) Sampled pedestrian positions (excluding pruned samples in the center) are drawn as points with their collision area, borders of the safe polytopes are drawn as colored lines. (c) Similar to (b) but where sampled trajectories are drawn as dashed lines and support constraints are highlighted.

TABLE III: Statistical results over 100 experiments of the marginal CP ("CP_k") and trajectory CP ("CP"), the task duration, traveled distance, minimum distance to the pedestrians and computation times for the unimodal simulation with 4 pedestrians. For the CPs we report the maximum observed over all experiments and compare it to the specified bound (a dash indicates that no bound is specified on the particular CP). Other results are reported as "average (standard deviation)" unless stated otherwise. Methods are grouped by their safety guarantee.

Method	Max CP_k/Spec. (%)	Max CP/Spec. (%)	Dur. [s]	Trav. [m]	Min Dist. [m]	Runtime (Max) [ms]
S-MPCC ($\epsilon_k = 0.05$)	0.0043/0.0500 (9)	0.0082/- (-)	5.56 (0.36)	8.84 (0.08)	0.27 (0.06)	19 (98)
Gaussian ($\epsilon_k = 0.05$)	0.0455/0.0500 (91)	0.1271/- (-)	5.27 (0.10)	8.82 (0.05)	0.13 (0.05)	14 (64)
S-MPCC ($\epsilon_k = 0.0025$)	0.0003/0.0025 (10)	0.0004/- (-)	6.39 (0.09)	9.00 (0.05)	0.39 (0.06)	20 (77)
Gaussian ($\epsilon_k = 0.0025$)	0.0025/0.0025 (99)	0.0069/- (-)	**5.33** (0.08)	**8.87** (0.04)	0.26 (0.06)	**12** (111)
SH-MPC ($\epsilon = 0.05$)	0.0070/- (-)	0.0113/0.0500 (23)	5.64 (0.40)	9.00 (0.10)	0.28 (0.05)	23 (61)

TABLE IV: Results for the unimodal simulation with 8 pedestrians. Displayed results follow the notation in Table III.

Method	Max CP_k/Spec. (%)	Max CP/Spec. (%)	Dur. [s]	Trav. [m]	Min Dist. [m]	Runtime (Max) [ms]
Gaussian ($\epsilon_k = 0.05$)	0.0865/0.0500 (173)	0.2309/- (-)	11.76 (3.07)	18.53 (0.43)	0.14 (0.05)	14 (73)
Gaussian ($\epsilon_k = 0.0025$)	0.0041/0.0025 (162)	0.0140/- (-)	16.11 (1.58)	19.11 (0.43)	0.30 (0.07)	13 (58)
Gaussian ($\epsilon_k = 0.0003125$)	0.0004/0.0025 (16)	0.0016/- (-)	18.25 (3.31)	**19.24** (0.42)	0.37 (0.07)	**14** (59)
SH-MPC ($\epsilon = 0.05$)	0.0047/- (-)	0.0128/0.0500 (26)	**15.63** (1.73)	19.32 (0.22)	0.34 (0.05)	30 (79)

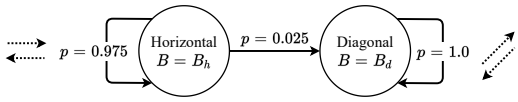


Fig. 10: Markov-chain representation of crossing pedestrian motion.

noise of the previous simulations. We define the pedestrian dynamics as

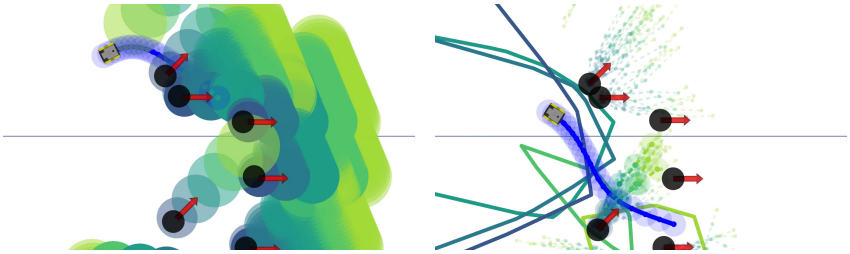
$$\boldsymbol{p}_{k+1} = \boldsymbol{p}_k + (B\boldsymbol{v} + \boldsymbol{\delta}_{w,k})dt, \quad \boldsymbol{\delta}_{w,k} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}_{w,k}), \quad (37)$$

where B is either $B_h = \begin{bmatrix}1 & 0\end{bmatrix}^T$ or $B_d = \begin{bmatrix}\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}}\end{bmatrix}^T$ depending on the state of the Markov Chain. The uncertainties associated with this motion can be modeled as a Gaussian Mixture Model (GMM), where each possible state transition in the Markov Chain leads to a separate mode with an associated probability (in total 21 modes). For example, the mode where the pedestrian crosses after two steps occurs with probability $p_2 = (1 - 0.025)0.025$. The setup of these simulations is visualized in Fig. 11. We apply the Gaussian baseline to the GMM distribution by formulating the constraints at a risk of ϵ_k for all modes, which can lead to conservatism when multiple modes influence the plan. Since previous simulations showed that S-MPCC performs worse than the other two methods, we do not include it in this comparison. We validate in this environment with 8 pedestrians.

Results are summarized in Table V. We again validate against the baseline with $\epsilon_k = 0.0003125$. SH-MPC outperforms the baselines on almost all metrics. We also note that the computation times of the Gaussian method are excessive due to the many modes to be considered, while the computation times of SH-MPC are unaffected. Some collisions occur for the higher risk methods in this environment. Collisions are observed when a pedestrian starts crossing towards the robot. In this case the predictions change between two instances of the control problem and make the optimization infeasible, freezing the robot in place.

F. Empirical Analysis and Sensitivity

Fig 13 visualizes the empirical risk distribution of the multimodal 8 pedestrian simulation (Sec. IX-E) with respect to the support of the SP (Eq. 3). This distribution empirically confirms that higher support leads to higher risk on average as well as longer tails.

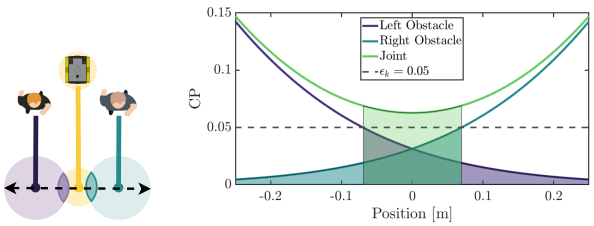


(a) Gaussian baseline at $\epsilon_k = 0.0003125$. (b) SH-MPC at $\epsilon = 0.05$.

Fig. 11: Snapshot of the simulation where pedestrian motion follows a GMM using the same visualizations as in Fig. 9. (a) Level sets of each mode are visualized. (b) Sampled trajectories consist of crossing and non-crossing behavior.

TABLE V: Results for the multimodal simulation with 8 pedestrians. Displayed results follow the notation in Table III, with minimum distance replaced by the number of collisions.

Method	**Max CP$_k$/Spec. (%)**	**Max CP/Spec. (%)**	**Dur. [s]**	**Trav. [m]**	**Collisions**	**Runtime (Max) [ms]**
Gaussian ($\epsilon_k = 0.05$)	0.1042/0.0500 (208)	0.3346/- (-)	16.98 (4.69)	20.27 (1.17)	8	144 (451)
Gaussian ($\epsilon_k = 0.0025$)	0.0050/0.0025 (198)	0.0215/- (-)	17.97 (4.94)	20.51 (1.09)	6	150 (447)
Gaussian ($\epsilon_k = 0.0003125$)	0.0006/0.0025 (24)	0.0028/- (-)	18.59 (5.32)	20.66 (1.11)	**0**	148 (409)
SH-MPC ($\epsilon = 0.05$)	0.0057/- (-)	0.0126/0.0500 (25)	**18.02** (3.66)	**18.74** (0.23)	3	**29** (71)



(a) Example case (b) Joint and marginalized risk in the direction of the black dashed arrow in (a).

Fig. 12: A 1D illustration of the case where two obstacles constrain the robot. Even though the marginal probability of collision for each obstacle is less than ϵ_k (shaded tails) in the center region, the joint probability of collision in the feasible region (green shaded area) is larger than ϵ_k.

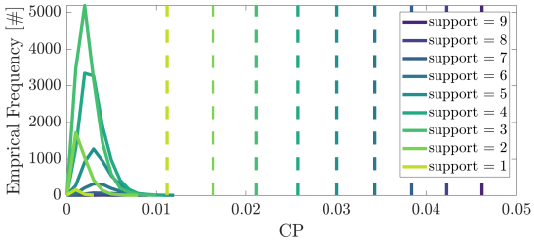


Fig. 13: The empirical distribution of the trajectory CP obtained by SH-MPC for each support value under pedestrian motion distributed as a Gaussian Mixture Model. Dashed vertical lines denote the theoretical CP bound for each support.

We validate the sensitivity of SH-MPC to varying risk specifications (ϵ) and horizon lengths (N) in the multimodal 8 pedestrian simulations. Results are depicted in Fig. 14. Fig. 14a shows that reducing the specified risk results in longer task durations. It also shows that the approach becomes more conservative for lower risk specifications. In accordance with our approach, Fig 14c indicates that changing the horizon length does not significantly affect the CP. Instead, the trajectories become more cautious when the same risk must be guaranteed over a longer duration. We observe in Fig. 14b and 14d that the computation times are mainly increasing when the specified risk is reduced. This is due to its relation with the sample size.

G. Autonomous Navigation in an Urban Environment

SH-MPC can be applied to different robot morphologies and scenarios. As demonstration, we deploy our approach on a simulated self-driving vehicle in Carla simulator [41]. The dynamics are modeled with a second order bicycle model [42] and the collision region consists of 3 discs. We construct a collision-free polytope for each of the discs. The pedestrians are programmed to follow the same dynamics as in Sec. IX-E, i.e., a GMM modeling crossing behavior. We do not model the interaction between the vehicle and the pedestrians. The control frequency is 10 Hz. We measured the computation time to be 88 ms on average and 135 ms maximum. Fig. 15 visualizes snapshots of the simulations. A video of the simulations is provided with this paper [40]. In Case A (see Fig. 15b), the planner keeps sufficient distance to let the pedestrians cross, while driving as close to the path as is safe. In Case B (see Fig. 15c) the vehicle passes behind the pedestrians while keeping distance to the pedestrian that is not crossing.

X. Discussion

As expected from the theoretical analysis, our experiments showed that SH-MPC can consistently bound the CP of the overall trajectory. Where methods that impose marginal constraints need to decide between performance ($\epsilon_k = \epsilon$) and safety ($\epsilon_k = \frac{\epsilon}{NM}$), SH-MPC makes this trade-off more explicit by ensuring that the CP remains consistent under different operating conditions, such as with regards to number of obstacles, probability distributions, and the horizon length.

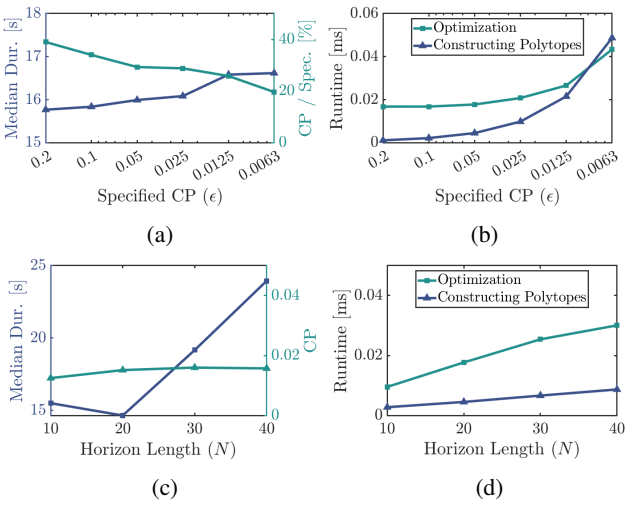
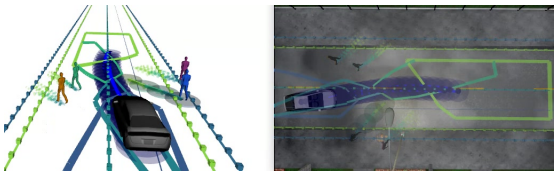
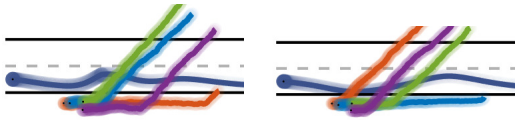


Fig. 14: Sensitivities of the empirical CP and runtimes with respect to the specified CP (ϵ) and the horizon length (N), evaluated over 10 experiments in the setting of Sec. IX-E. Task duration is denoted "Dur.". With "CP" we denote the mean over the experiments of the maximum trajectory CP. The runtimes are separated in solving the optimization and computing free-space polytopes (30) from the scenarios.



(a) A snapshot of the proposed approach in the Carla simulator.



(b) Case A (c) Case B

Fig. 15: (a) Snapshot from Carla simulations with visualizations as in Fig. 11b for the frontal vehicle disc in stages $0, 6, 12$ and 19. Plots (b) and (c) show the observed trajectories of the vehicle (dark blue) and pedestrians (other colors) in two cases.

The gap between the risk guarantee and the obtained risk can be reduced, for example, by assuming some knowledge of the distribution or by running multiple scenario programs in parallel. Alternatively, the risk could be analyzed in continuous time (see for example [43]) to reduce discretization errors.

The proposed method is widely applicable. It handles dynamic obstacles (e.g., cyclists, cars or non-cooperative robots) and controls systems with nonlinear dynamics. In addition, the joint distribution of the uncertainty can capture interactions of dynamic obstacles with other obstacles or the robot (e.g., to predict that pedestrians evade other pedestrians and the vehicle). It cannot yet account for interaction during planning, where the dynamics of the robot and pedestrians directly influence each other, as the probability measure $\mathbb{P}$ cannot depend on the optimization variables in scenario optimization. The guarantees provided in this paper rely on an accurate model of the uncertainty, which may be challenging to obtain, for example in the case of human motion prediction. Nevertheless, the proposed method provides a planner that attains a desired level of risk with respect to the predicted probability distribution. The prediction model could also be replaced by recorded samples, in line with the typical scenario approach, to reduce modeling errors and provide formal guarantees with respect to the true motion of the obstacles.

In terms of computational efficiency, we showed that our approach is online capable and scalable under typical operating conditions. For extreme low risk specifications (e.g., $\epsilon \leq 5{\cdot}10^{-3}$), computational requirements may become excessive as a result of the increase in sample size. This can be addressed, for example, by either pruning the samples, given that only the extreme samples are of interest (see for example [10]), or by solving an approximate scenario optimization [44]. In addition, most computations of SH-MPC are parallel linear computations (for each sample), which potentially leave room for further optimization, e.g., by delegating computations to a Graphical Processing Unit (GPU).

XI. Conclusions

We presented a novel method for planning under uncertainty, Safe Horizon Model Predictive Control (SH-MPC), that bounds the collision probability of the planned trajectory over its duration and with respect to all obstacles. The method uses a scenario optimization formulation where samples of the involved uncertainty are used as constraints to limit the collision probability of the motion plan. The number of samples is the main indicator for the collision probability and under our approach could be computed before deploying the controller.

Our simulations, with a mobile robot and an autonomous vehicle, showed that SH-MPC better approximates the collision probability over the duration of the motion plan than existing methods that rely on the marginal probability of collision. The main baseline, that achieved tight evaluations of the risk for each time step, was shown to be conservative over the duration of its motion plan and there was significant variation in its overall risk when we varied the number of obstacles or their distribution. The overall risk of SH-MPC remained less conservative and more consistent between different environments and distributions, which resulted in faster trajectories when the environment was crowded. In addition, we showed excellent scaling of the computation time with respect to the number of obstacles and under varying distributions.

Acknowledgments

The authors would like to thank Bruno Brito for assisting in the simulated experiments.

REFERENCES

[1] M. Simon, Inside the Amazon Warehouse Where Humans and Machines Become One, 2019. [Online]. Available: https://www.wired.com/story/amazon-warehouse-robots/

[2] J. Walker, The Self-Driving Car Timeline - Predictions from the Top 11 Global Automakers, 2019. [Online]. Available: https://emerj.com/ai-adoption-timelines/self-driving-car-timeline-themselves-top-11-automakers/

[3] J. F. P. Kooij, F. Flohr, E. A. I. Pool, and D. M. Gavrila, "Context-Based Path Prediction for Targets with Switching Dynamics," Int. J. of Computer Vision, vol. 127, no. 3, pp. 239–262, Mar. 2019.

[4] H. Zhu and J. Alonso-Mora, "Chance-Constrained Collision Avoidance for MAVs in Dynamic Environments," IEEE Robot. Autom. Lett., vol. 4, no. 2, pp. 776–783, Apr. 2019.

[5] B. Luders, M. Kothari, and J. How, "Chance Constrained RRT for Probabilistic Robustness to Environmental Uncertainty," in Proc. AIAA Guid., Navigat. Control Conf., Toronto, Ontario, Canada, Aug. 2010.

[6] L. Blackmore, Hui Li, and B. Williams, "A probabilistic approach to optimal robust path planning with obstacles," in Proc. Amer. Control Conf., Jun. 2006, pp. 2831–2837.

[7] J. van den Berg, P. Abbeel, and K. Goldberg, "LQG-MP: Optimized path planning for robots with motion uncertainty and imperfect state information," Int. J. Robot. Res., vol. 30, no. 7, pp. 895–913, Jun. 2011.

[8] A. Wang, A. Jasour, and B. C. Williams, "Non-Gaussian Chance-Constrained Trajectory Planning for Autonomous Vehicles Under Agent Uncertainty," IEEE Robot. Autom. Lett., vol. 5, no. 4, pp. 6041–6048, Oct. 2020, conference Name: IEEE Robotics and Automation Letters.

[9] A. Wang, X. Huang, A. Jasour, and B. Williams, "Fast Risk Assessment for Autonomous Vehicles Using Learned Models of Agent Futures," arXiv:2005.13458 [cs, stat], Jun. 2020.

[10] O. de Groot, B. Brito, L. Ferranti, D. Gavrila, and J. Alonso-Mora, "Scenario-Based Trajectory Optimization in Uncertain Dynamic Environments," IEEE Robot. Autom. Lett., pp. 5389 – 5396, 2021.

[11] M. C. Campi, S. Garatti, and F. A. Ramponi, "A General Scenario Theory for Nonconvex Optimization and Decision Making," IEEE Trans. Automat. Contr., vol. 63, no. 12, pp. 4067–4078, Dec. 2018.

[12] B. Paden, M. Čáp, S. Z. Yong, D. Yershov, and E. Frazzoli, "A Survey of Motion Planning and Control Techniques for Self-Driving Urban Vehicles," IEEE Trans. on Intelligent Vehicles, vol. 1, no. 1, pp. 33–55, Mar. 2016.

[13] W. Schwarting, J. Alonso-Mora, and D. Rus, "Planning and Decision-Making for Autonomous Vehicles," Annu. Rev. Control Robot. Auton. Syst., vol. 1, no. 1, pp. 187–210, May 2018.

[14] W. Schwarting, J. Alonso-Mora, L. Paull, S. Karaman, and D. Rus, "Safe Nonlinear Trajectory Generation for Parallel Autonomy With a Dynamic Vehicle Model," IEEE Trans. Intelligent Transportation Systems, vol. 19, no. 9, pp. 2994–3008, Sep. 2018.

[15] B. Brito, B. Floor, L. Ferranti, and J. Alonso-Mora, "Model Predictive Contouring Control for Collision Avoidance in Unstructured Dynamic Environments," IEEE Robot. Autom. Lett., vol. 4, no. 4, pp. 4459–4466, Oct. 2019.

[16] A. Ben-Tal and A. Nemirovski, "Robust Convex Optimization," Math. Oper. Res., vol. 23, no. 4, pp. 769–805, Nov. 1998.

[17] A. Mesbah, "Stochastic Model Predictive Control: An Overview and Perspectives for Future Research," IEEE Control Systems Magazine, vol. 36, no. 6, pp. 30–44, Dec. 2016.

[18] B. Kouvaritakis and M. Cannon, Model Predictive Control: Classical, Robust and Stochastic, ser. Advanced Textbooks in Control and Signal Processing. Springer International Publishing, 2016.

[19] J. F. Fisac, A. Bajcsy, S. L. Herbert, D. Fridovich-Keil, S. Wang, C. J. Tomlin, and A. D. Dragan, "Probabilistically Safe Robot Planning with Confidence-Based Human Predictions," arXiv:1806.00109 [cs], May 2018.

[20] Masahiro Ono and B. C. Williams, "Iterative Risk Allocation: A new approach to robust Model Predictive Control with a joint chance constraint," in 47th IEEE Conf. on Decision and Control, Dec. 2008, pp. 3427–3432.

[21] S. Patil, J. v. d. Berg, and R. Alterovitz, "Estimating probability of collision for safe motion planning under Gaussian motion and sensing uncertainty," in IEEE Int. Conf. Robot. Autom., May 2012, pp. 3238–3244.

[22] L. Blackmore, M. Ono, A. Bektassov, and B. C. Williams, "A Probabilistic Particle-Control Approximation of Chance-Constrained Stochastic Predictive Control," IEEE Trans. Robot., vol. 26, no. 3, pp. 502–517, Jun. 2010.

[23] L. Janson, E. Schmerling, and M. Pavone, "Monte Carlo Motion Planning for Robot Trajectory Optimization Under Uncertainty," arXiv:1504.08053 [cs], May 2015.

[24] E. Schmerling and M. Pavone, "Evaluating Trajectory Collision Probability through Adaptive Importance Sampling for Safe Motion Planning," arXiv:1609.05399 [cs], Jun. 2017.

[25] G. S. Aoude, B. D. Luders, J. M. Joseph, N. Roy, and J. P. How, "Probabilistically safe motion planning to avoid dynamic obstacles with uncertain motion patterns," Autonomous Robots, vol. 35, no. 1, pp. 51–76, Jul. 2013.

[26] K. Ren, H. Ahn, and M. Kamgarpour, "Chance-Constrained Trajectory Planning With Multimodal Environmental Uncertainty," IEEE Control Systems Letters, vol. 7, pp. 13–18, 2023, conference Name: IEEE Control Systems Letters.

[27] G. Calafiore and M. Campi, "The scenario approach to robust control design," IEEE Trans. Automat. Contr., vol. 51, no. 5, pp. 742–753, May 2006.

[28] M. C. Campi and S. Garatti, "The Exact Feasibility of Randomized Solutions of Uncertain Convex Programs," SIAM Journal on Optimization, vol. 19, no. 3, pp. 1211–1230, Jan. 2008.

[29] ——, "A Sampling-and-Discarding Approach to Chance-Constrained Optimization: Feasibility and Optimality," Journal of Optimization Theory and Applications, vol. 148, no. 2, pp. 257–280, Feb. 2011.

[30] G. Schildbach, L. Fagiano, and M. Morari, "Randomized Solutions to Convex Programs with Multiple Chance Constraints," SIAM Journal on Optimization, vol. 23, no. 4, pp. 2479–2501, Jan. 2013.

[31] S. Garatti and M. C. Campi, "Risk and complexity in scenario optimization," Math. Program., vol. 191, pp. 243–279, Nov. 2019.

[32] P. Billingsley, Probability and measure, 3rd ed., ser. Wiley series in probability and mathematical statistics. New York: Wiley, 1995.

[33] J. Nocedal and S. J. Wright, Numerical optimization, 2nd ed., ser. Springer series in operations research. New York: Springer, 2006.

[34] G. C. Calafiore, "Repetitive Scenario Design," IEEE Transactions on Automatic Control, vol. 62, no. 3, pp. 1125–1137, Mar. 2017, conference Name: IEEE Transactions on Automatic Control.

[35] K. A. Mustafa, O. de Groot, X. Wang, J. Kober, and J. Alonso-Mora, "Probabilistic Risk Assessment for Chance-Constrained Collision Avoidance in Uncertain Dynamic Environments," Feb. 2023. [Online]. Available: http://arxiv.org/abs/2302.10846

[36] O. de Groot, "Jupyter Notebook for Scenario Dimensioning," Apr. 2022. [Online]. Available: https://colab.research.google.com/drive/1Xye7960CZAlt4gHpoH6HOvok4-YuLFbD?usp=sharing

[37] A. Domahidi and J. Jerez, FORCES Professional. Embotech AG, Jul. 2014. [Online]. Available: https://embotech.com/FORCES-Pro

[38] F. J. A. Artacho, R. Campoy, and M. K. Tam, "The Douglas-Rachford Algorithm for Convex and Nonconvex Feasibility Problems," arXiv:1904.09148 [math], Apr. 2019.

[39] R. Siegwart and I. R. Nourbakhsh, Introduction to Autonomous Mobile Robots, 2nd ed. The MIT Press, 2011.

[40] O. de Groot, L. Ferranti, D. Gavrila, and J. Alonso-Mora, "Scenario-Based Motion Planning with Bounded Probability of Collision," Jun. 2022. [Online]. Available: https://youtu.be/-rqoTICmEO4

[41] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An Open Urban Driving Simulator," in Conf. on Robot Learning, Oct. 2017, pp. 1–16.

[42] J. Kong, M. Pfeiffer, G. Schildbach, and F. Borrelli, "Kinematic and dynamic vehicle models for autonomous driving control design," in IEEE IV Symposium, Jun. 2015, pp. 1094–1099.

[43] K. M. Frey, T. J. Steiner, and J. P. How, "Collision Probabilities for Continuous-Time Systems Without Sampling [with Appendices]," Jun. 2020, arXiv:2006.01109 [cs, math]. [Online]. Available: http://arxiv.org/abs/2006.01109

[44] H. Sartipizadeh and B. Açikmeşe, "Approximate convex hull based scenario truncation for chance constrained trajectory optimization," Automatica, 2020.



Oscar de Groot received both the B.Sc. degree in electrical engineering, in 2016, and the M.Sc. degree in systems & control, in 2019, from the Delft University of Technology, Delft, The Netherlands. He is currently pursuing a Ph.D. in motion planning for autonomous vehicles in urban environments at the department of Cognitive Robotics at the Delft University of Technology. His research interests include probabilistic safe motion planning, scenario optimization, model predictive control and self-driving vehicles.



Laura Ferranti received her PhD from Delft University of Technology, Delft, The Netherlands, in 2017. She is currently an assistant professor in the Cognitive Robotics (CoR) Department, Delft University of Technology, Delft, The Netherlands. She is the recipient of an NWO Veni Grant from The Netherlands Organisation for Scientific Research (2020), and of the Best Paper Award in Multi-robot Systems at ICRA 2019. Her research interests include optimization and optimal control, model predictive control, reinforcement learning, embedded optimization-based control with application in flight control, maritime transportation, robotics, and automotive.



Dariu Gavrila received the Ph.D. degree in computer science from Univ. of Maryland at College Park, USA, in 1996. From 1997 until 2016, he was with Daimler R&D, Ulm, Germany, where he became a Distinguished Scientist. He led the vision-based pedestrian detection research, which was commercialized 2013-2014 in various Mercedes-Benz models. In 2016, he moved to TU Delft, where he since heads the Intelligent Vehicles group as a Full Professor. His current research deals with sensor-based detection of humans and analysis of behavior in the context of self-driving vehicles. He was awarded the Outstanding Application Award 2014 and the Outstanding Researcher Award 2019, both from the IEEE Intelligent Transportation Systems Society.



Javier Alonso-Mora is an Associate Professor at the Cognitive Robotics department of the Delft University of Technology, and a Principal Investigator at the Amsterdam Institute for Advanced Metropolitan Solutions (AMS Institute). Before joining TU Delft, Dr. Alonso-Mora was a Postdoctoral Associate at the Massachusetts Institute of Technology (MIT). He received his Ph.D. degree in robotics from ETH Zurich,

His main research interest is in navigation, motion planning and control of autonomous mobile robots, with a special emphasis on multi-robot systems, on-demand transportation and robots that interact with other robots and humans in dynamic and uncertain environments. He is the recipient of an ERC Starting Grant (2021), the ICRA Best Paper Award on Multi-Robot Systems (2019), an Amazon Research Award (2019) and a talent scheme VENI Grant from the Netherlands Organisation for Scientific Research (2017).

APPENDIX

In practice, linearization of the collision avoidance constraints results in a support that consist of a small subset of the scenarios. To support this observation, we consider here how the constraint formulation impacts the support.

Definition 2. The *shadow* of a scenario $\boldsymbol{\delta}^{(i)}$ under the constraints $\boldsymbol{\theta} \in \Theta_{\delta^{(i)}}$ is a region $S_{\boldsymbol{\delta}^{(i)}} \subseteq \Delta$ such that if another scenario satisfies $\boldsymbol{\delta} \in S_{\delta^{(i)}}$, then either $\boldsymbol{\delta}^{(i)}$ or $\boldsymbol{\delta}$ is redundant. That is,

$$S_{\boldsymbol{\delta}^{(i)}} = \{\boldsymbol{\delta} \in \Delta \mid \Theta_{\boldsymbol{\delta}^{(i)}} \subset \Theta_{\boldsymbol{\delta}}\} \cup \{\boldsymbol{\delta} \in \Delta \mid \Theta_{\boldsymbol{\delta}} \subset \Theta_{\boldsymbol{\delta}^{(i)}}\}.$$

An example shadow is visualized in Fig. 16a for the proposed constraints. Note that in Fig. 16b, the shadow always occupies a non zero area. This is a result of the linearization and a set of box constraints on the position that limit the range in which constraints are considered. In this case we can prove the following.

Theorem 6. Suppose that the shadow is non-empty for all possible samples in the support (i.e., $S_{\boldsymbol{\delta}} \neq \emptyset,\ \forall \boldsymbol{\delta} \in \Delta$), then the probability that out of S samples, none of the samples are redundant goes to 0 exponentially, for $S \to \infty$.

Proof. Given S samples, a new sample $\boldsymbol{\delta}$ is not redundant if it falls outside of the aggregated shadow, i.e., if $\boldsymbol{\delta} \notin \bigcup_{i=1}^{S} S_{\boldsymbol{\delta}^{(i)}}$. The associated probability is

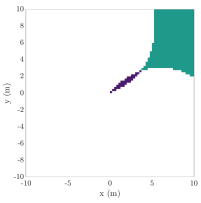
$$P_S = \mathbb{P}\left[\boldsymbol{\delta} \in \Delta \mid \boldsymbol{\delta} \notin \bigcup_{i=1}^{S} S_{\boldsymbol{\delta}^{(i)}}\right]. \tag{38}$$

Since $S_{\boldsymbol{\delta}} \neq \emptyset,\ \forall \boldsymbol{\delta} \in \Delta$, the aggregated shadow grows when sample $S+1$ is not redundant, i.e., $\bigcup_{i=1}^{S} S_{\boldsymbol{\delta}^{(i)}} \subset \bigcup_{i=1}^{S+1} S_{\boldsymbol{\delta}^{(i)}}$. Therefore, $P_{S+1} < P_S$ and additionally $P_i < 1$ for $i > 0$. We obtain the following result,

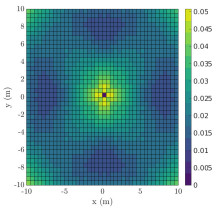
$$\lim_{S\to\infty} \mathbb{P}^S[\text{No redundant scenarios}] = \lim_{S\to\infty} \prod_{i=1}^{S} P_i = 0, \tag{39}$$

implying that the probability of sampling no redundant scenarios goes to zero. This probability is upper bounded by $(P_1)^S$, i.e., it converges at least exponentially. □

Although this does not prove that the support is bounded for finite S, it shows that it is very likely that redundant scenarios are sampled, which are not of support.



(a) Shadow for a sample at $(2.5, 3.5)$ in blue (dominated by) and green (dominates).



(b) The area occupied by the shadow for each point within the box-constraints.

Fig. 16: Empirical 2-D visualizations of the shadow region for linearized constraints (27).